

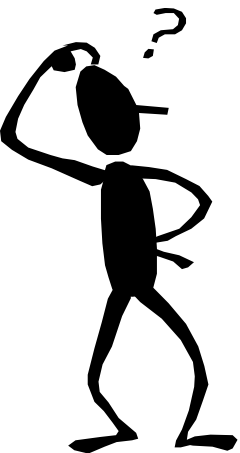
软件构造

-----第2章 软件详细设计

主讲：彭辉

Email: penghui@cuit.edu.cn

本章目标



- 1.了解详细设计概念和内容
- 2.掌握结构化详细设计基本方法
- 3.掌握面向对象详细设计基本方法
- 4.掌握撰写详细设计说明书

第2章 软件详细设计

- 2.1 详细设计概念和内容
- 2.2 结构化详细设计
- 2.3 McCabe程序复杂度
- 2.4 面向对象详细设计
- 2.5 人机界面设计
- 2.6 详细设计案例

2.1详细设计概念和内容

2.1.1详细设计概念



类似于建造房子，概要设计定义了各个房间的功能，以及房间之间的连接要素，详细设计则对各房间进行详细描述，也即设计每个模块的实现算法、所需的局部数据结构等。

2.1.2详细设计内容

详细设计

模块的功能描述

模块的接口描述(包括UI)

模块的数据描述

模块的类、程序流程描述

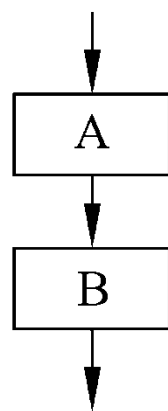
模块的算法描述

2.2结构化详细设计

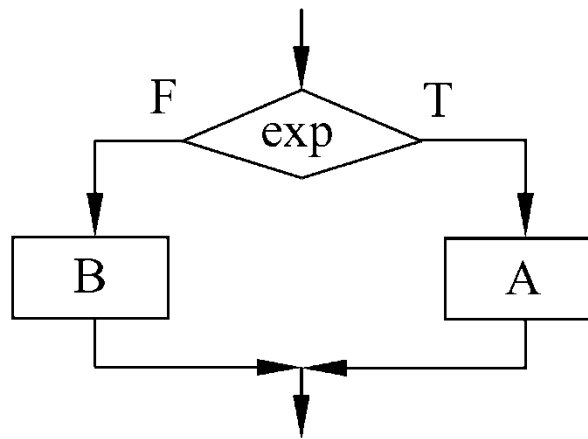
- **详细设计目标：**确定应该怎样具体地实现所要求的系统。经过这个阶段的设计工作，应该得出对目标系统的精确描述，从而在编码阶段可以把这个描述直接翻译成用某种程序设计语言书写的程序。
- 结构化程序设计技术是实现上述目标的关键技术，因此是详细设计的逻辑基础。

2.2.1 结构程序设计

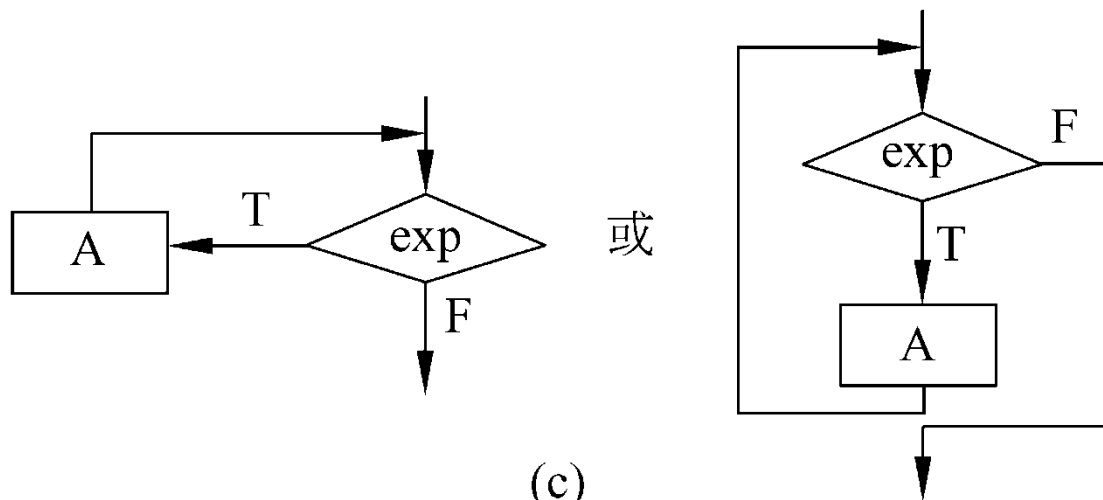
- **经典定义：**如果一个程序的代码块仅仅通过顺序、选择和循环这3种基本控制结构进行连接，并且每个代码块只有一个入口和一个出口，则称这个程序是结构化的。
- **更全面定义：**结构程序设计是尽可能少用GOTO语句的程序设计方法。最好仅在检测出错误时才使用GOTO语句，而且应该总是使用前向GOTO语句。



(a)

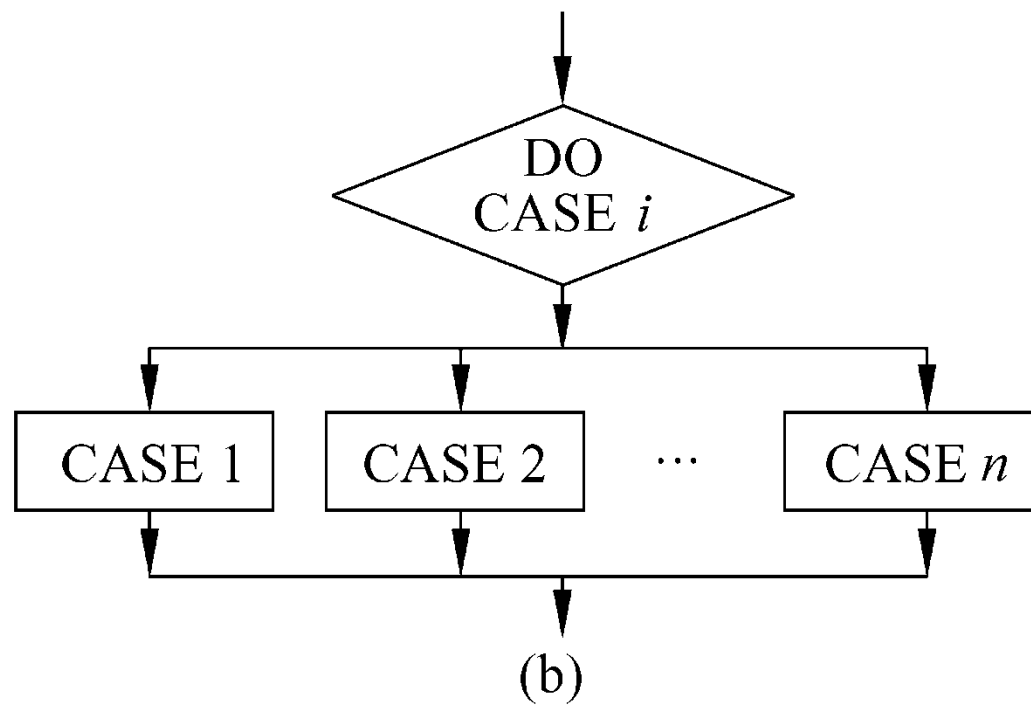
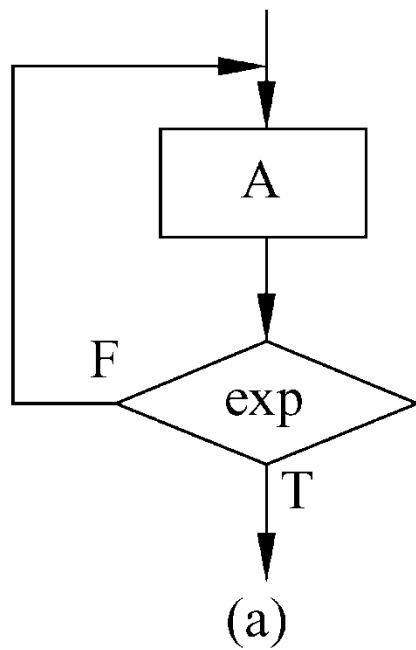


(b)



(c)

3种基本的控制结构



其他常用的控制结构

- **经典的结构程序设计：**只允许使用顺序、**IF-THEN-ELSE**型分支和**DO-WHILE**型循环这3种基本控制结构；
- **扩展的结构程序设计：**如果除了上述3种基本控制结构之外，还允许使用**DO-CASE**型多分支结构和**DO-UNTIL**型循环结构；
- **修正的结构程序设计：**再加上允许使用**LEAVE(或BREAK)**结构。

2.2.1 结构化详细设计工具

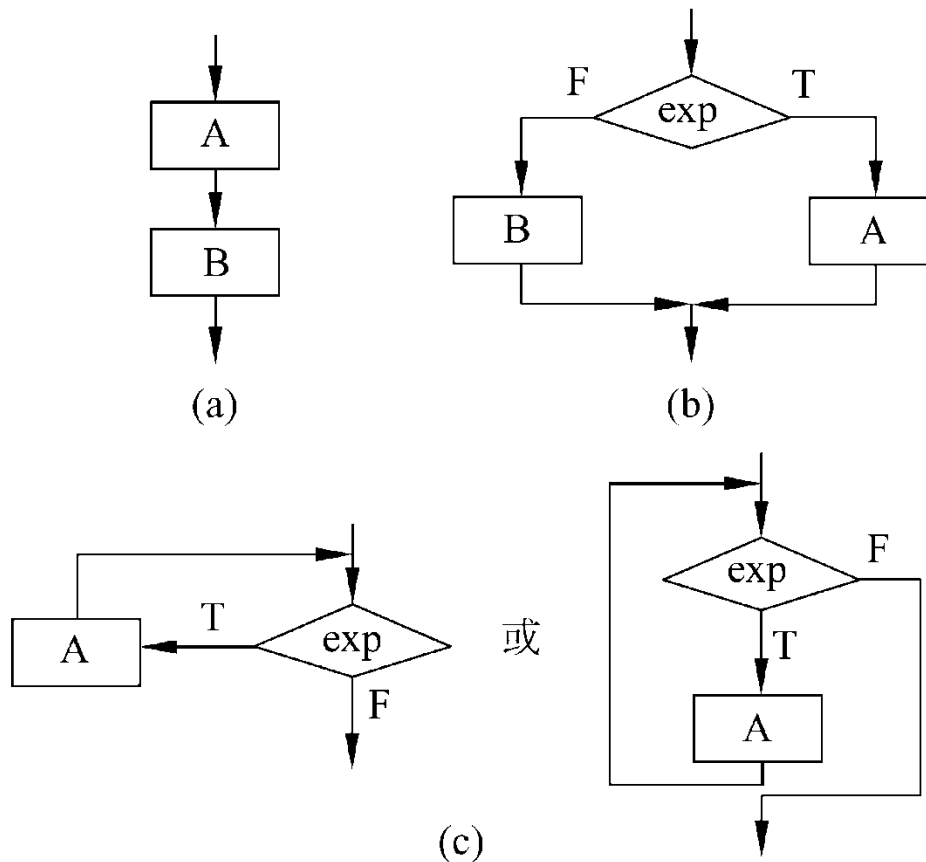
- 程序流程图的表示方式
- N-S图描述算法
- PAD图描述算法
- 表格的设计方式（判定表和判定树）
- 过程设计语言PDL

2.2.2 程序流程图

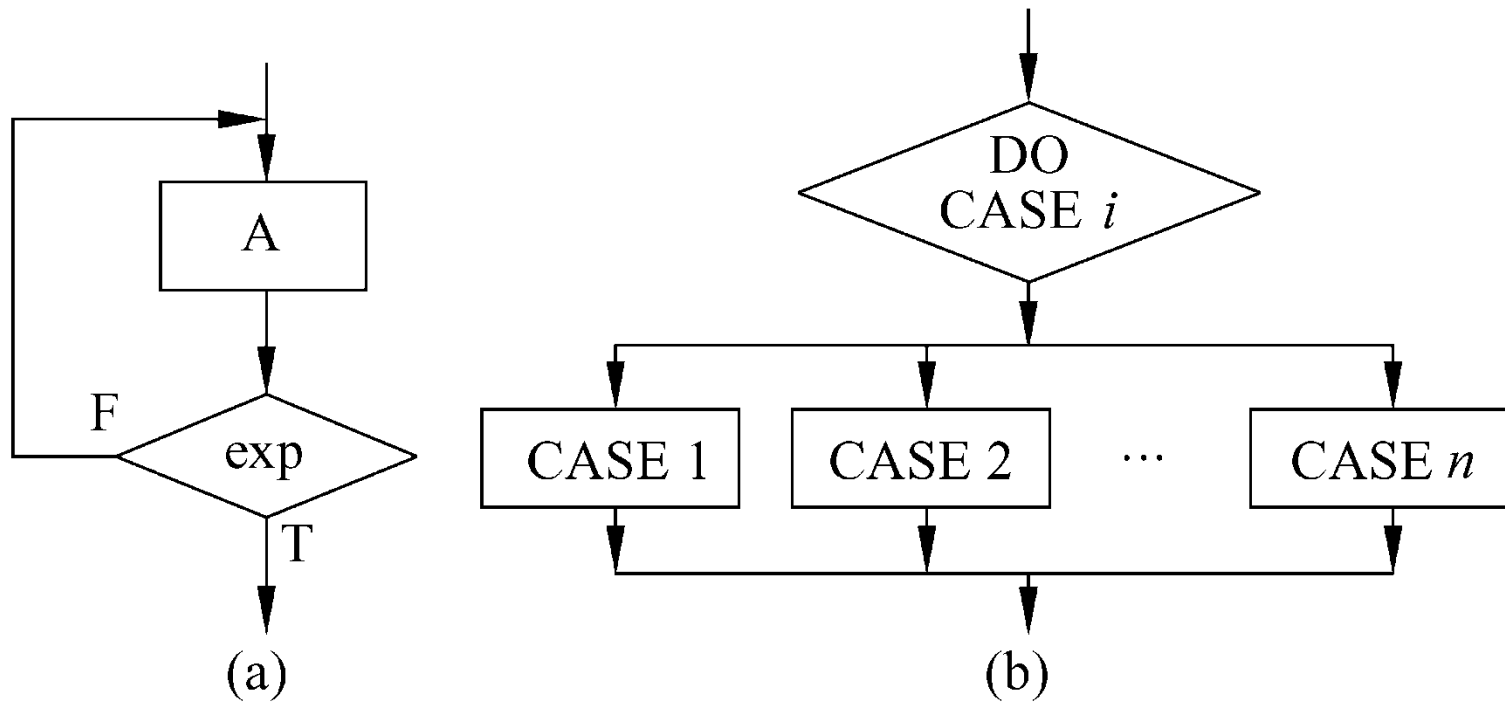
- 程序流程图又称为程序框图，它是历史最悠久、使用最广泛的描述过程设计的方法，至今仍在广泛使用。
- 它的主要优点是对控制流程的描绘很直观，便于初学者掌握。

2.2.2 程序流程图

程序流程图的表示方式



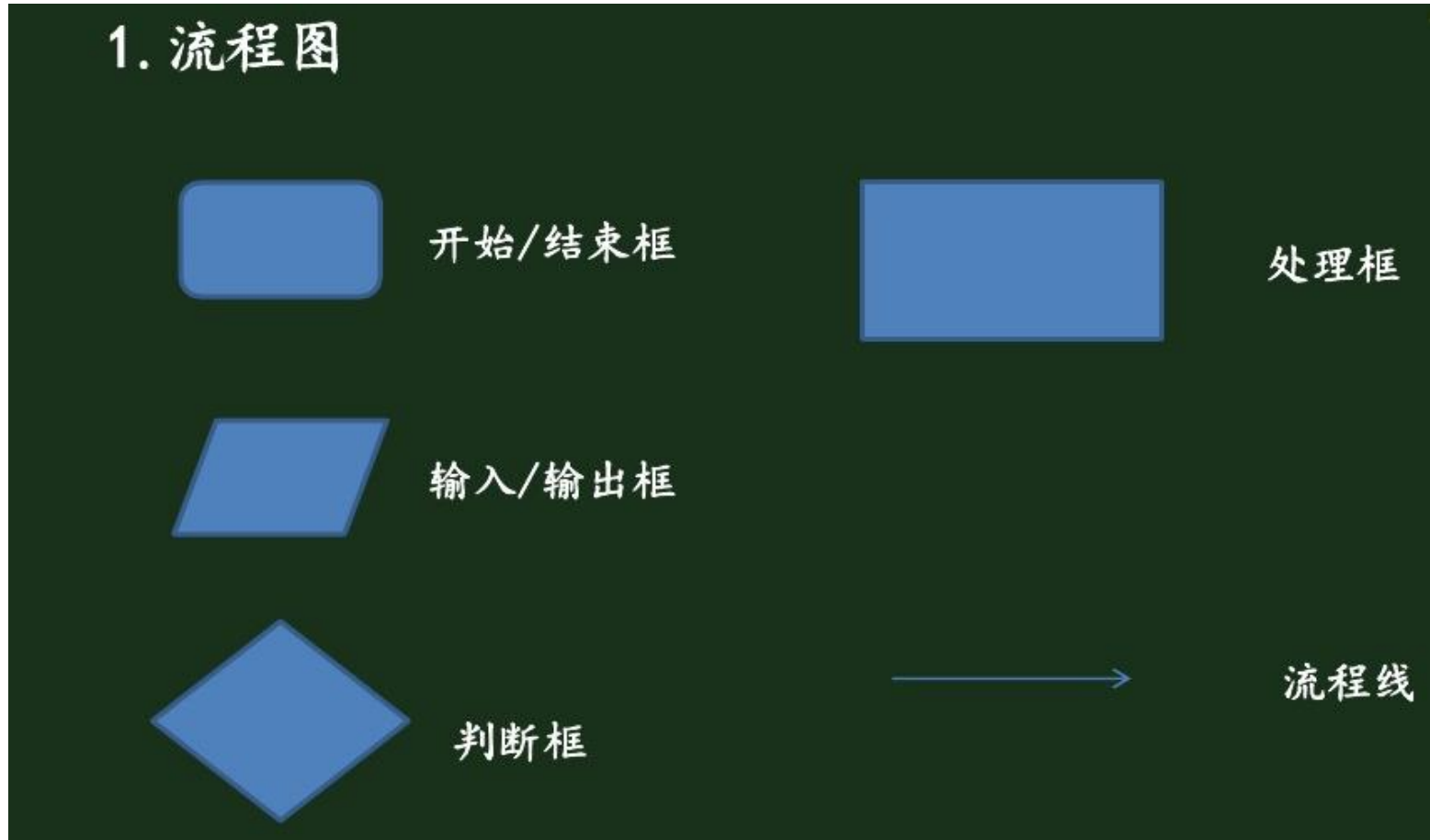
2.2.2 程序流程图



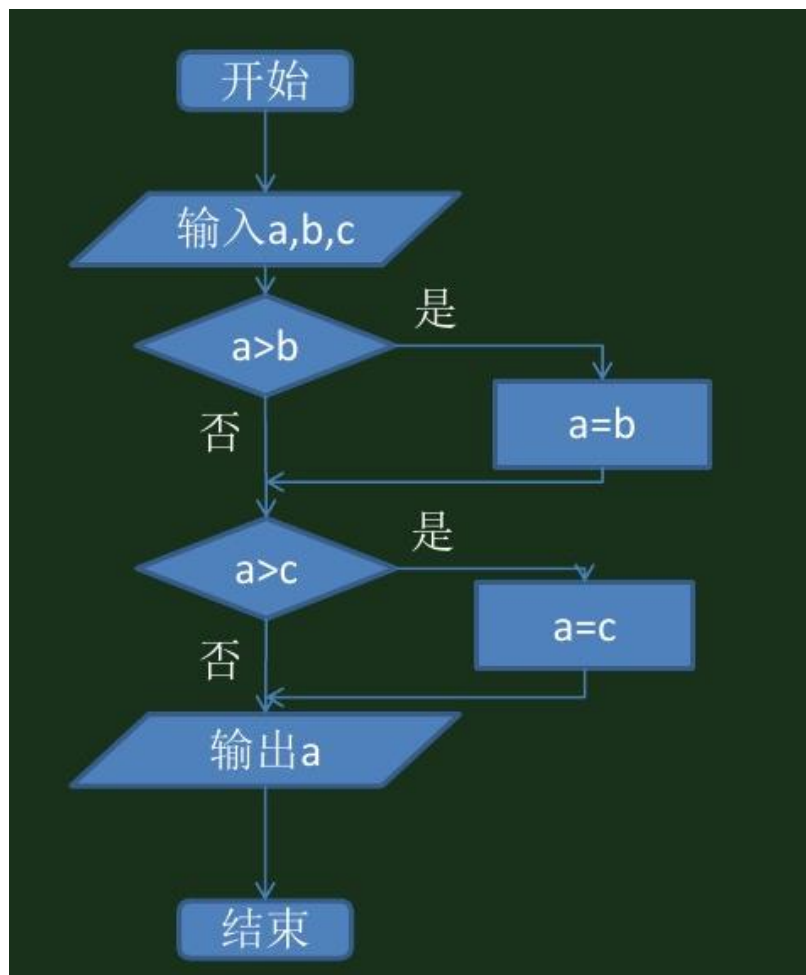
其他常用的控制结构

2.2.2 程序流程图

■ 流程图常用符号

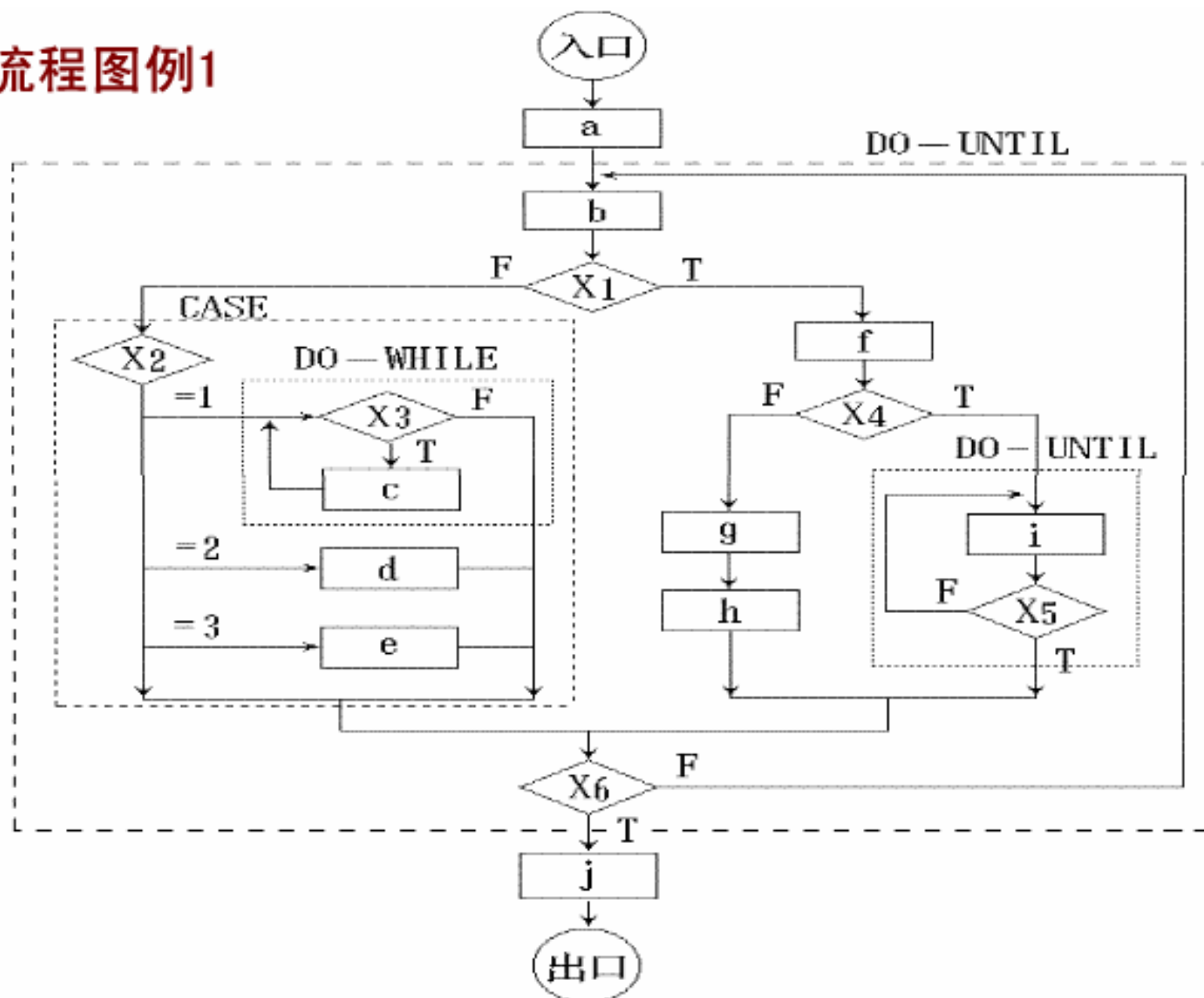


2.2.2 程序流程图



求a、b、c三数中的最小数

流程图例1



程序流程图的主要缺点：

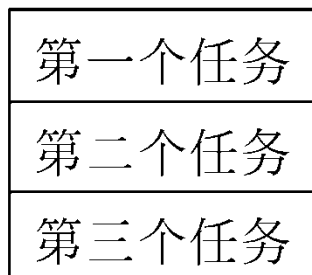
- 程序流程图本质上不是逐步求精的好工具，它诱使程序员过早地考虑程序的控制流程，而不去考虑程序的全局结构。
- 程序流程图中用箭头代表控制流，因此程序员不受任何约束，可以完全不顾结构程序设计的精神，随意转移控制。
- 程序流程图不易表示数据结构。

2.2.3 盒图(N-S图)

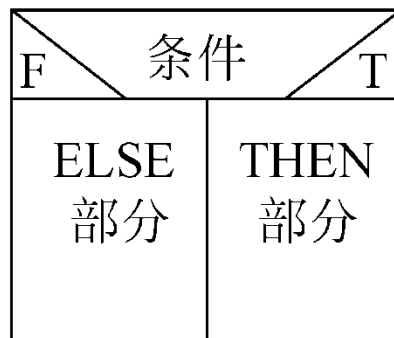
盒图具有下述特点：

- 功能域明确。
 - 不可能任意转移控制。
 - 很容易确定局部和全程数据的作用域。
 - 很容易表现嵌套关系，也可以表示模块的层次结构。
-

2.2.2 盒图(N-S图)



(a)



(b)



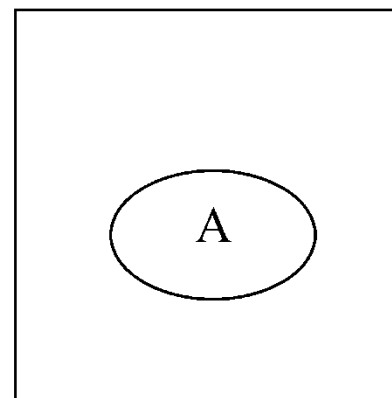
(c)



(d)



(d)

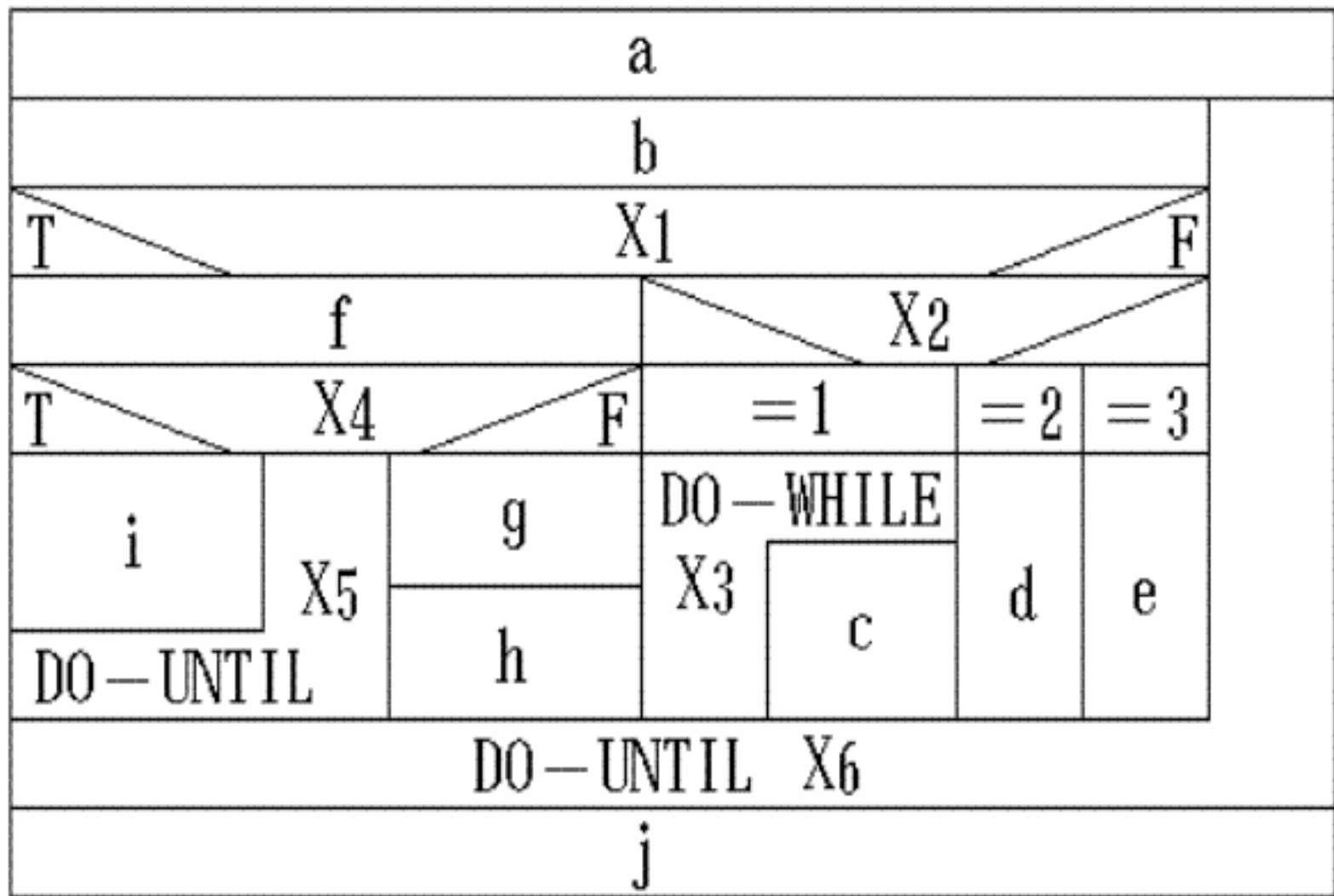


(e)

盒图的基本符号

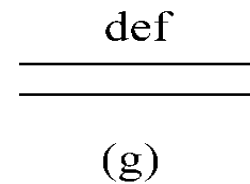
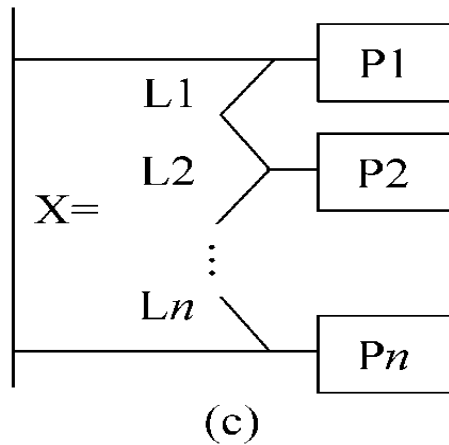
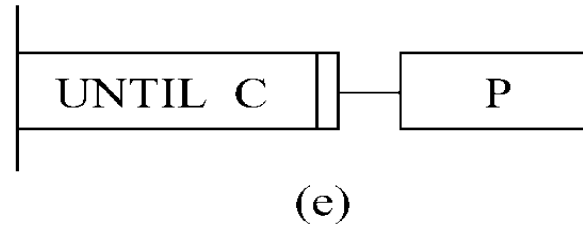
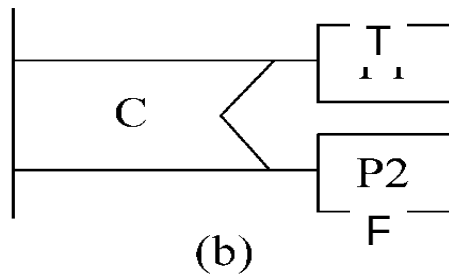
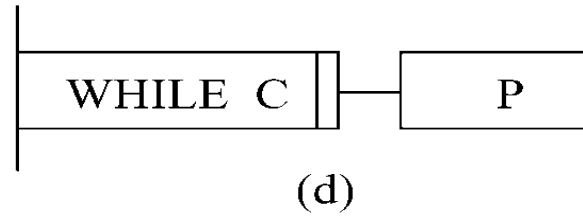
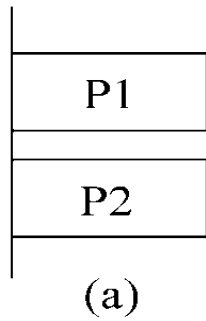
(a)顺序、(b)if分支、(c)Case分支、(d)循环、(e)调用子程序A

2.2.3 盒图(N-S图)



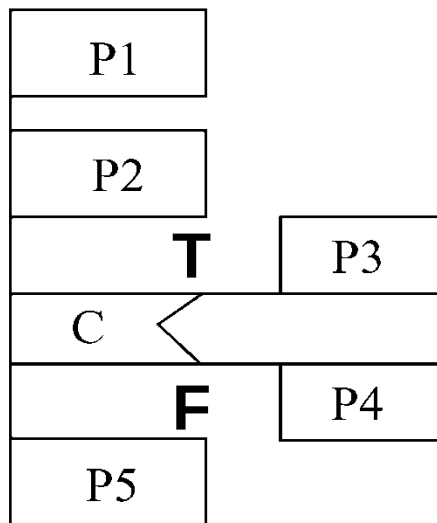
2.2.4 PAD图

- **PAD**是问题分析图(problem analysis diagram)的英文缩写，自1973年由日本日立公司发明以后，已得到一定程度的推广。
- 它用二维树形结构的图来表示程序的控制流，将这种图翻译成程序代码比较容易。
- 即可表示程序逻辑，也可描绘数据结构。
- **PAD**图的符号支持自顶向下、逐步求精方法的使用。

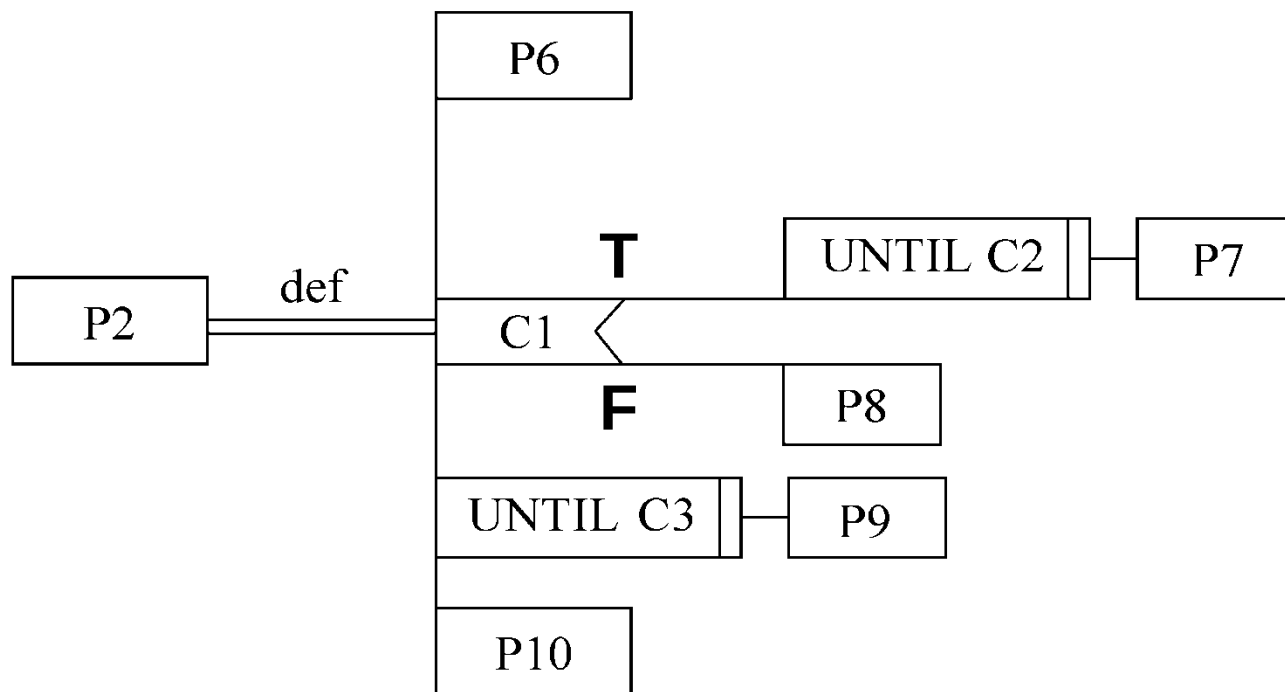


PAD图的基本符号

(a)顺序、(b)if分支、(c)Case分支、(d) (e)循环、(f)语句标号、(g)定义

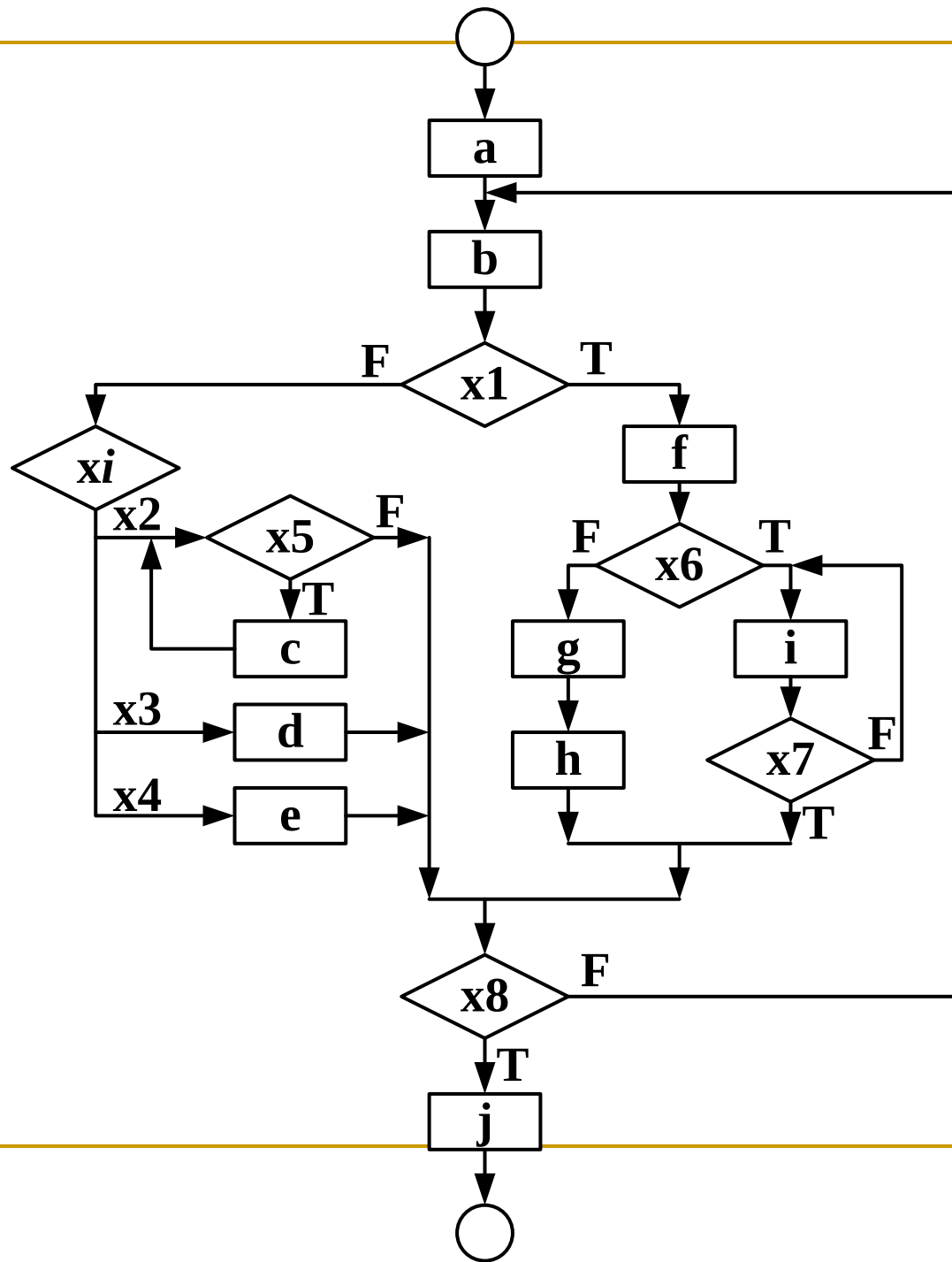


(a)

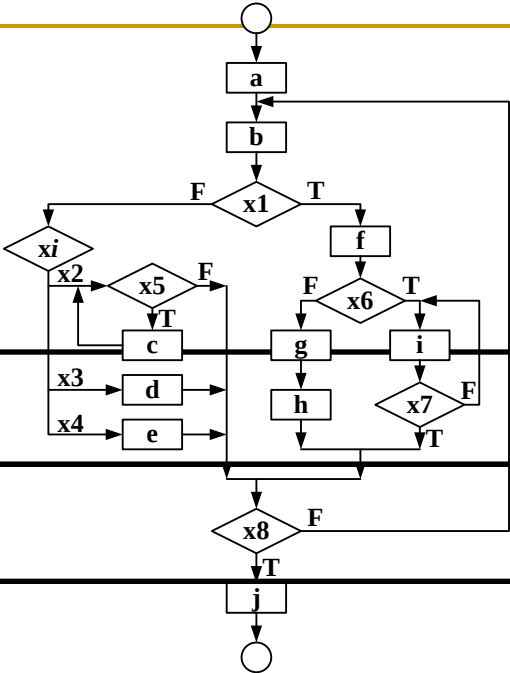


(b)

- **例题：**某程序流程图如右图所示，请分别用N-S图和PAD图表示。

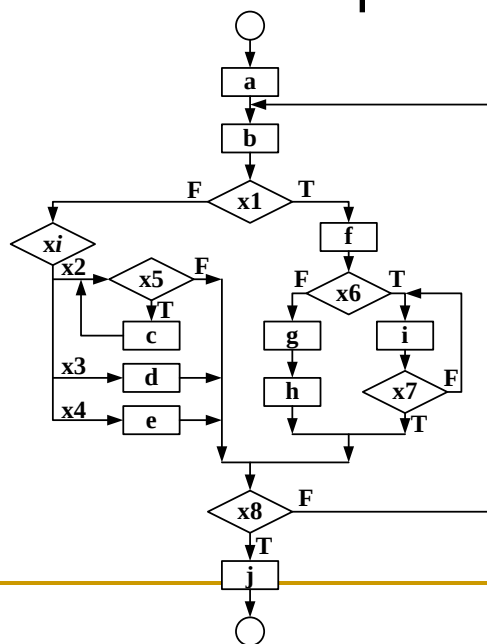
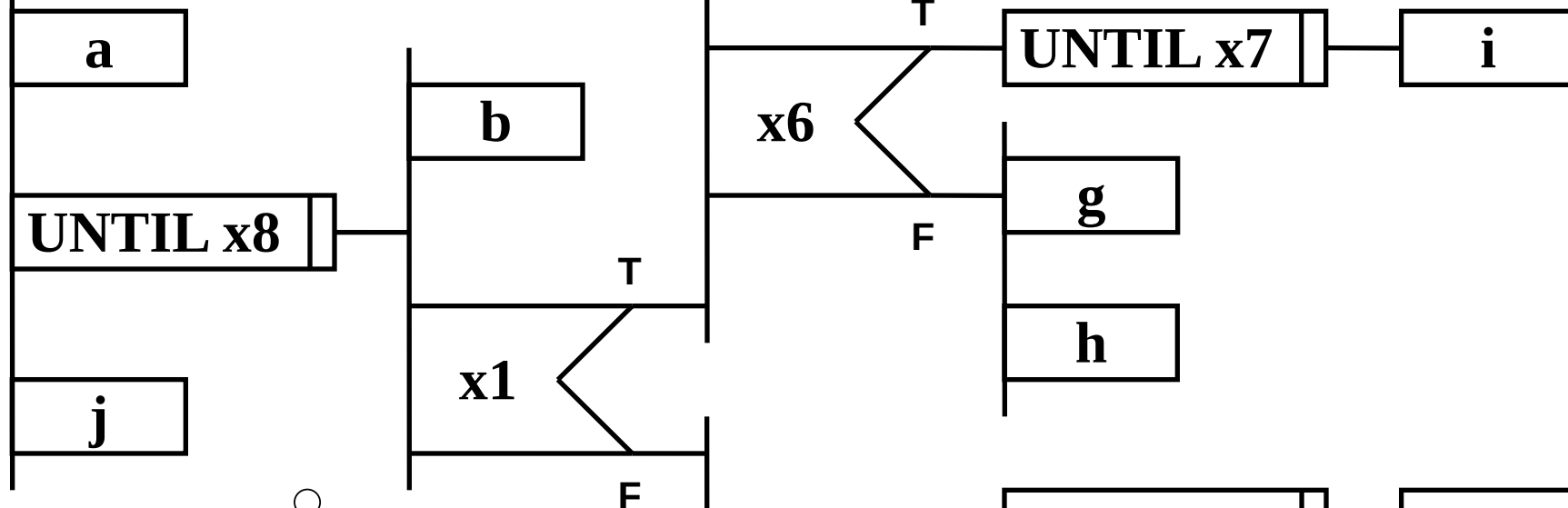


N-S图:



a			<pre>graph TD x3 --> d x4 --> e d --> h e --> h h --> x7{x7} x7 -- F --> i x7 -- T --> x8{x8} i --> x8 x8 -- F --> j x8 -- T --> end(())</pre>	
b				
x1				
CASE xi			f	
x2	x3	x4	x6	
x5	d	e	g	i
c			h	x7
x8				
j				

PAD图:



2.2.5 过程设计语言

- 过程设计语言(PDL)也称为伪码，它是用正文形式表示数据和处理过程的设计工具。
- PDL具有严格的关键字外部语法，用于定义控制结构和数据结构；另一方面，PDL表示实际操作和条件的内部语法通常又是灵活自由的，可以适应各种工程项目的需要。
- PDL是一种“混杂”语言，它使用一种语言的词汇，同时却使用另一种语言的语法

伪代码的基本控制结构:

- 顺序结构。

- 判定结构:

IF <条件>

一条或者多条语句

ELSEIF <条件>

一条或者多条语句

.....

ELSE

一条或者多条语句

ENDIF

■ 循环结构:

WHILE循环结构:

DO WHILE <条件>

一条或者多条语句

ENDWHILE

UNTIL循环结构:

REPEAT UNTIL <条件>

一条或者多条语句

ENDREP

FOR循环结构:

DOFOR <循环变量>=<循环变量取值范围, 表达式或者序列>

一条或者多条语句

ENDFOR

■ 子过程

PROCEDURE <子过程名> <属性表>

INTERFACE <参数表>

一条或者多条语句

END

例：检查订货单

```
IF 客户订货金额超过5000元 THEN
    IF 客户拖延未还赊欠钱款超过60天 THEN
        在偿还欠款前不予批准
    ELSE (拖延未还赊欠钱款不超过60天)
        发批准书，发货单
    ENDIF
ELSE (客户订货金额未超过5000元)
    IF 客户拖延未还赊欠钱款超过60天 THEN
        发批准书，发货单
        WHILE i < 3 DO
            发催款邮件
            i++
        END DO
    ELSE (拖延未还赊欠钱款不超过60天)
        发批准书，发货单
    ENDIF
ENDIF
```


PDL的优点：

- 可以作为注释直接插在源程序中间。有助于保持文档和程序的一致性，提高了文档的质量。
- 可以使用普通的正文编辑程序或文字处理系统，很方便地完成PDL的书写和编辑工作。
- 已经有自动处理程序存在，而且可以自动由PDL生成程序代码。

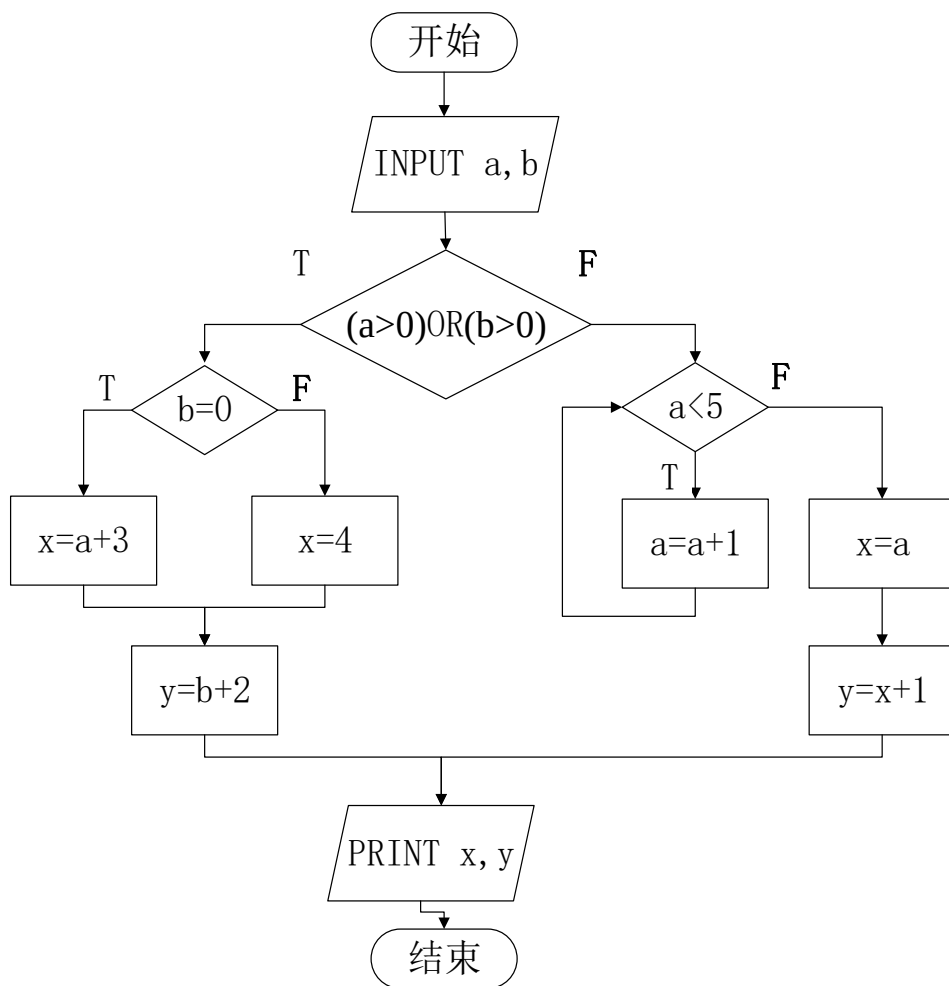
PDL的缺点：

- 不如图形工具形象直观，描述复杂的条件组合与动作间的对应关系时，不如判定表清晰简单。

- 课堂测验1:
- 画出下面用PDL写的流程图

```
START↵  
  INPUT(a,b)↵  
  IF (a>0)OR(b>0)↵  
  THEN ↵  
    IF b=0 THEN x=a+3 ELSE x=4 ENDIF↵  
    y=b+2↵  
  ELSE↵  
    WHILE a<5 DO ↵  
      a=a+1↵  
    END DO↵  
    x=a↵  
    y=x+1↵  
  END IF↵  
  PRINT(X,Y)↵  
STOP↵
```

■ 课堂测验1:

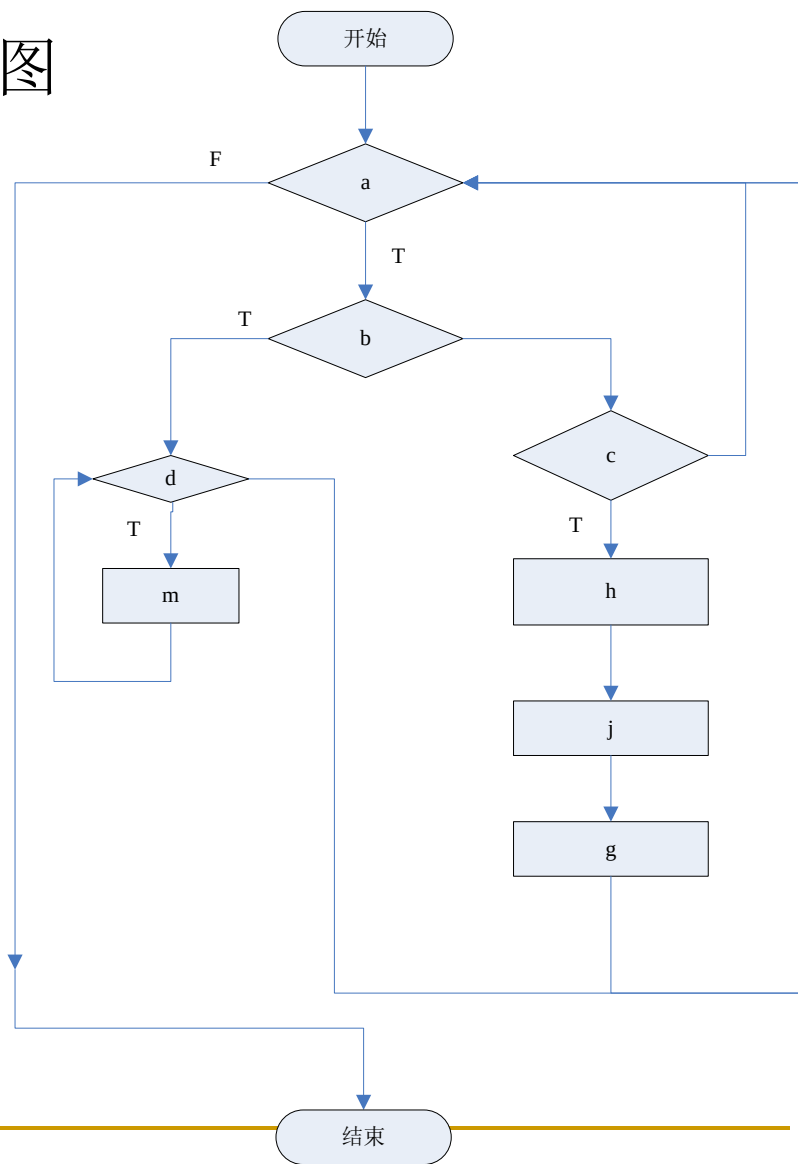


```
START↵  
  INPUT(a,b)↵  
  IF (a>0)OR(b>0)↵  
    THEN ↵  
      IF b=0 THEN x=a+3 ELSE x=4 ENDIF↵  
      y=b+2↵  
    ELSE↵  
      WHILE a<5 DO ↵  
        a=a+1↵  
      END DO↵  
      x=a↵  
      y=x+1↵  
    END IF↵  
  PRINT(X,Y)↵  
STOP↵
```

■ 课堂测验2:

- 画出下列伪码程序的程序流程图

```
START
WHILE a DO
  IF b THEN
    WHILE d DO
      m
    END DO
  ELSE
    IF c THEN
      BLOCK
        h
        j
        g
      END BLOCK
    END IF
  END IF
END DO
```

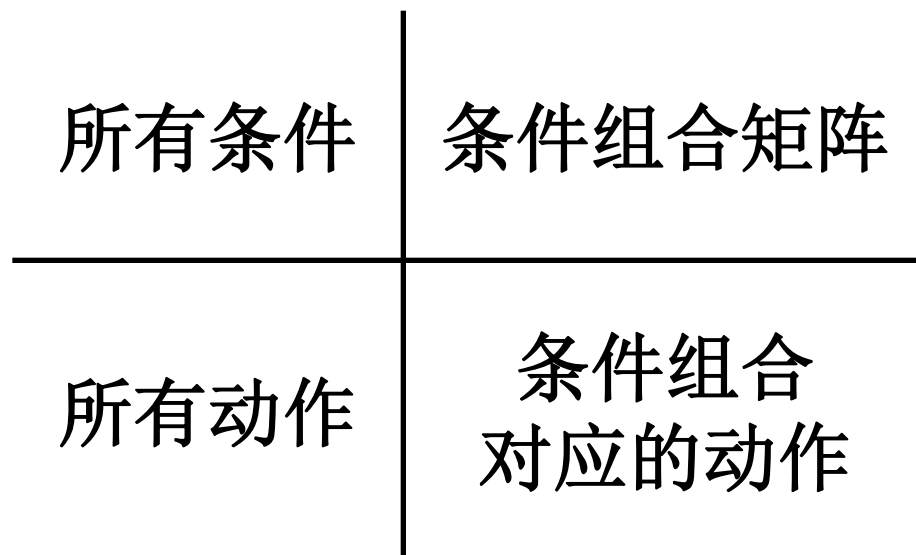


2.2.6 判定表

- 当算法中包含多重嵌套的条件选择时，用程序流程图、盒图、PAD图或后面即将介绍的过程设计语言(PDL)都不易清楚地描述。
- 判定表却能够清晰地表示复杂的条件组合与应做的动作之间的对应关系。

一张判定表由4部分组成：

- 左上部列出所有条件；
- 左下部是所有可能做的动作；
- 右上部是表示各种条件组合的一个矩阵；
- 右下部是和每种条件组合相对应的动作。



例题：

- 假设某航空公司规定，乘客可以免费托运重量不超过30kg的行李。
- 当行李重量超过30kg时，对头等舱的国内乘客超重部分每公斤收费4元，对其他舱的国内乘客超重部分每公斤收费6元。
- 对外国乘客超重部分每公斤收费比国内乘客多一倍，对残疾乘客超重部分每公斤收费比正常乘客少一半。

判定表的优点：

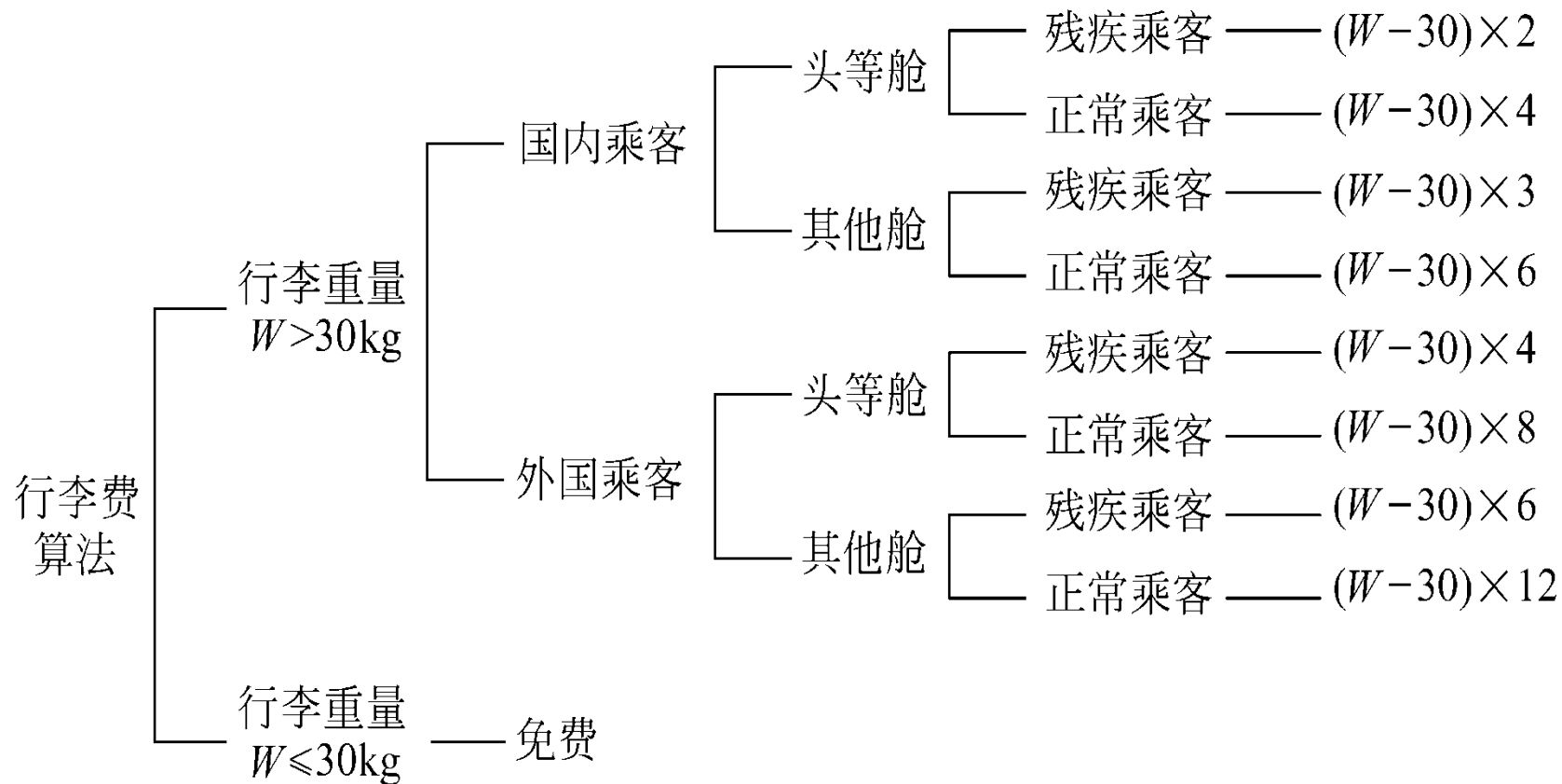
- 能清晰地表示复杂的条件组合与应做的动作之间的对应关系。

判定表的缺点：

- 判定表的含义不是一眼就能看出来的，初次接触这种工具的人理解它需要有一个简短的学习过程。
- 当数据元素的值多于两个时，判定表的简洁程度也将下降。

2.2.7 判定树

- 判定树是判定表的变种，也能清晰地表示复杂的条件组合与应做的动作之间的对应关系。
- 多年来判定树一直受到人们的重视，是一种比较常用的系统分析和设计的工具。



用判定树表示计算行李费的算法

判定树的优点：

- 它的形式简单，一眼就可以看出其含义，因此易于掌握和使用。

判定树的缺点：

- 简洁性不如判定表，数据元素的同一个值往往要重复写多遍，而且越接近树的叶端重复次数越多。

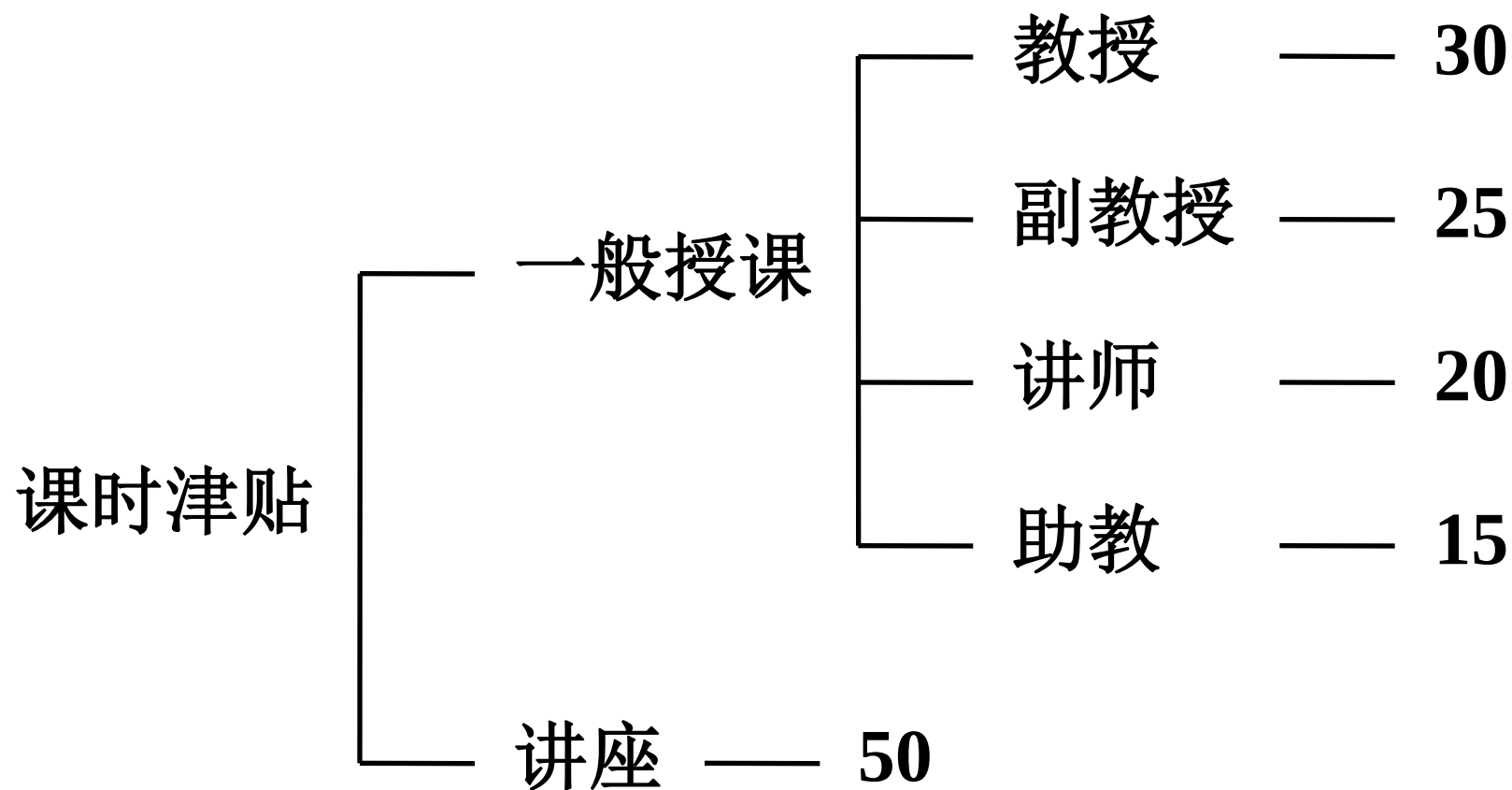
例题1:

- 某校制定了教师的讲课课时津贴标准。对于各种性质的讲座，无论教师是什么职称，每课时津贴费一律是50元；
- 对于一般的授课，则根据教师的职称来决定每课时津贴费：教授30元，副教授25元，讲师20元，助教15元。
- 请分别用判定表和判定树表示津贴标准。

判定表:

	1	2	3	4	5
教授		T	F	F	F
副教授		F	T	F	F
讲师		F	F	T	F
助教		F	F	F	T
讲座	T	F	F	F	F
50	√				
30		√			
25			√		
20				√	
15					√

判定树：



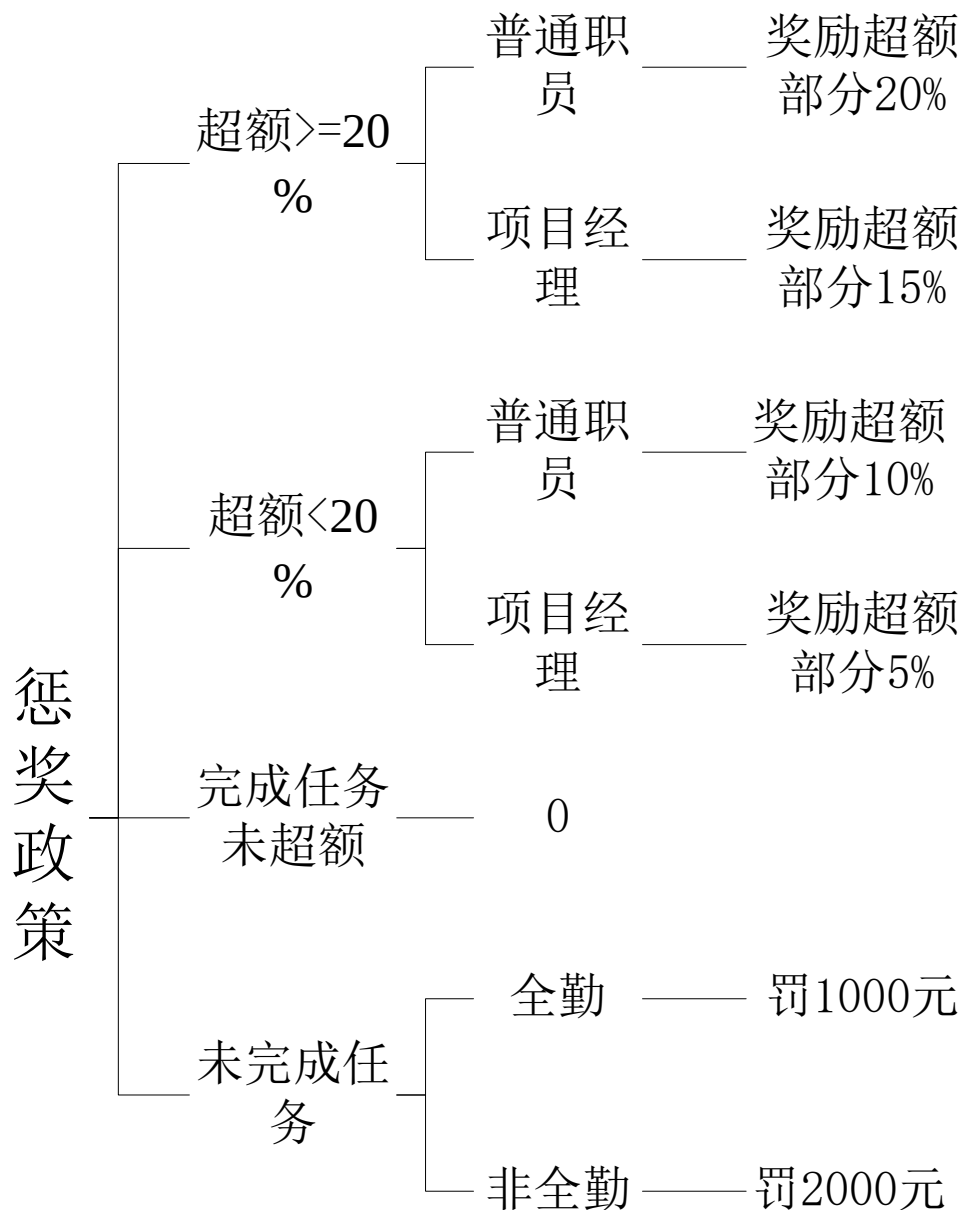
例题2:

- 某企业对员工工作进行考核，奖惩政策如下，请画出判定表和判定树。
 - a) 超额完成任务20%及以上，普通职员超额部分20%作为奖励，项目经理超额部分15%作为奖励。
 - b) 超额完成任务不足20%，普通职员超额部分10%作为奖励，项目经理超额部分5%作为奖励。
 - c) 完成任务但未超额，没有奖励。
 - d) 未完成任务者，若当月全勤罚款1000元，否则罚款2000元。

判定表:

	1	2	3	4	5	6	7
超额>20%	T	T	F	F	F	F	F
超额>0% && <=20%	F	F	T	T	F	F	F
完成任务未超额=0%	F	F	F	F	T	F	F
未完成任务超额<0%	F	F	F	F	F	T	T
普通职员	T	F	T	F			
全勤						T	F
奖励20%	√						
奖励15%		√					
奖励10%			√				
奖励5%				√			
0					√		
罚1000						√	
罚2000							√

判定树:



2.3 McCabe程序复杂度

- 把程序的复杂程度乘以适当常数即可估算出软件中错误的数量以及软件开发需要用的工作量;
- 定量度量的结果可以用来比较两个不同的设计或两个不同算法的优劣;
- 程序的定量的复杂程度可以作为模块规模的精确限度。

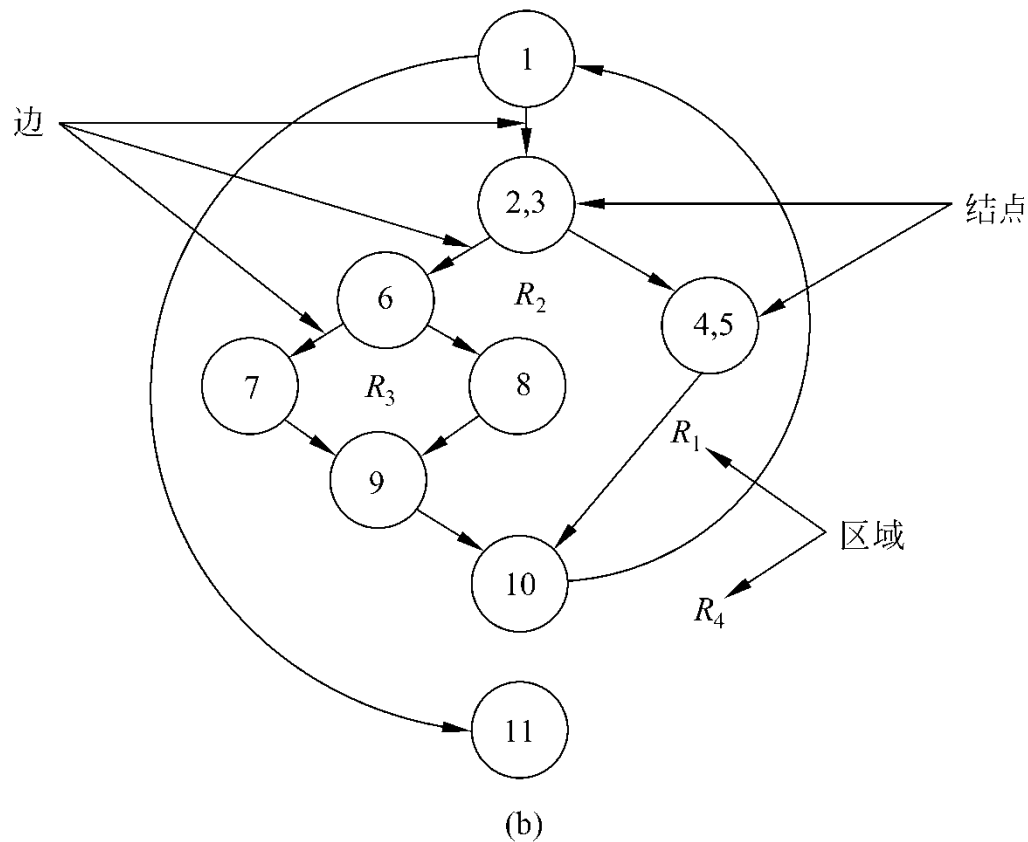
McCabe方法

1. 流图

- **McCabe**方法根据程序控制流的复杂程度定量度量程序的复杂程度，这样度量出的结果称为程序的环形复杂度。
- 所谓流图实质上是“退化了的”程序流程图，它仅仅描绘程序的控制流程，完全不表现对数据的具体操作以及分支或循环的具体条件。

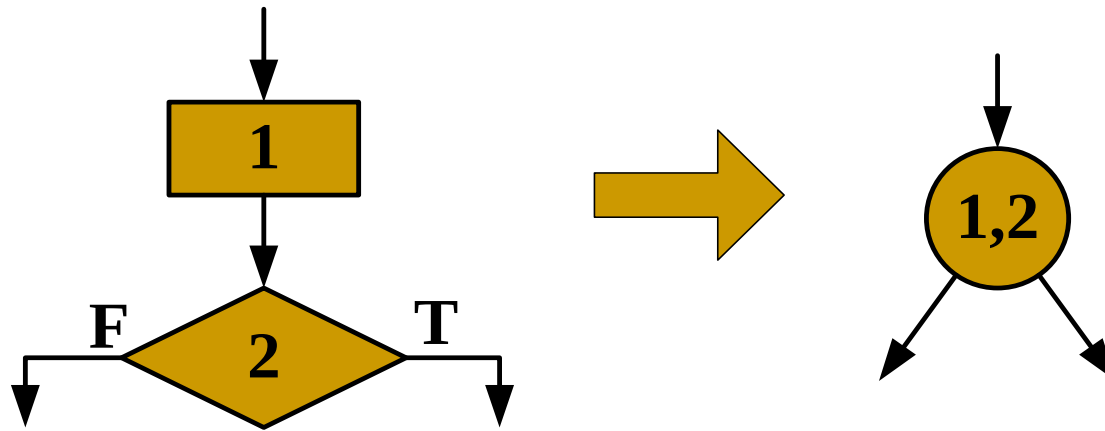
流图的表示:

- **结点**: 用圆表示, 一个圆代表一条或多条语句。
- **边**: 箭头线称为边, 代表控制流。在流图中一条边必须终止于一个结点, 即使这个结点并不代表任何语句。
- **区域**: 由边和结点围成的面积称为区域, 包括图外部未被围起来的区域。

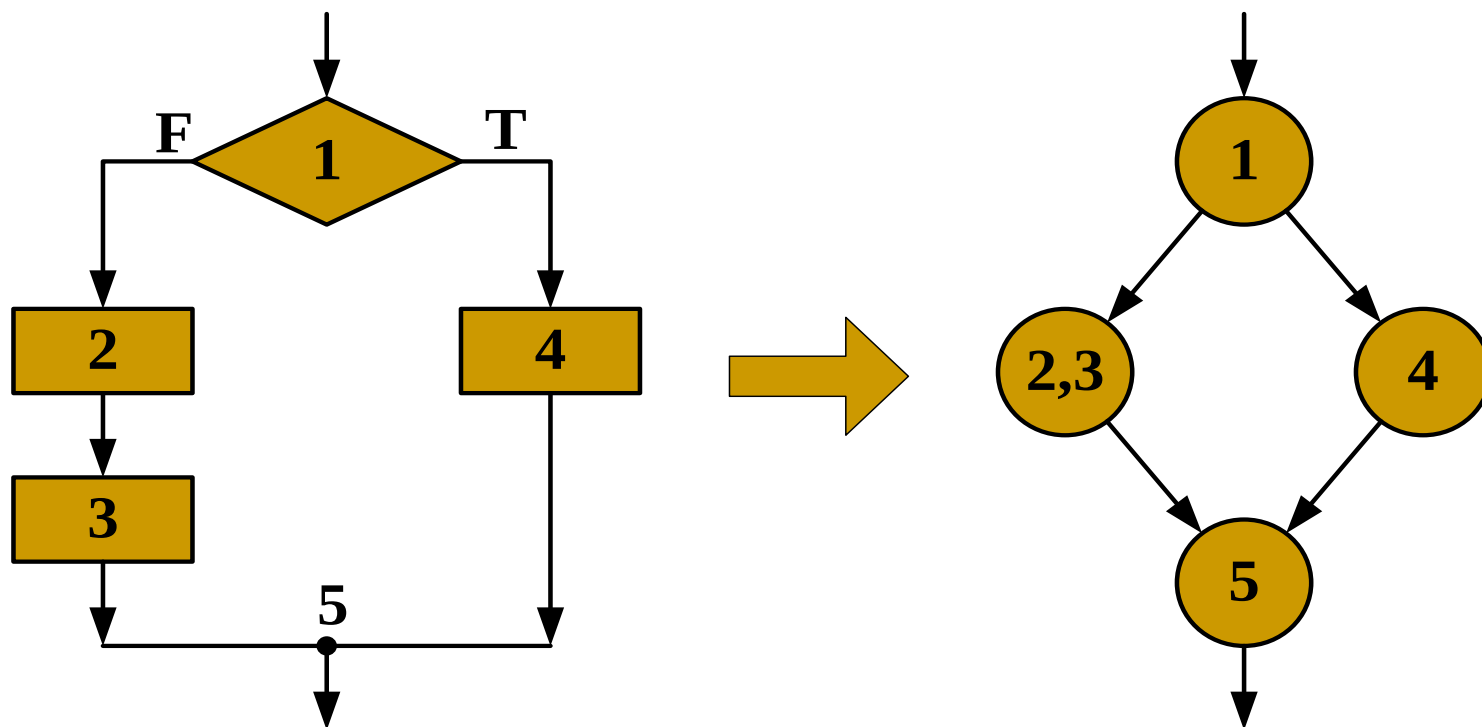


映射方法:

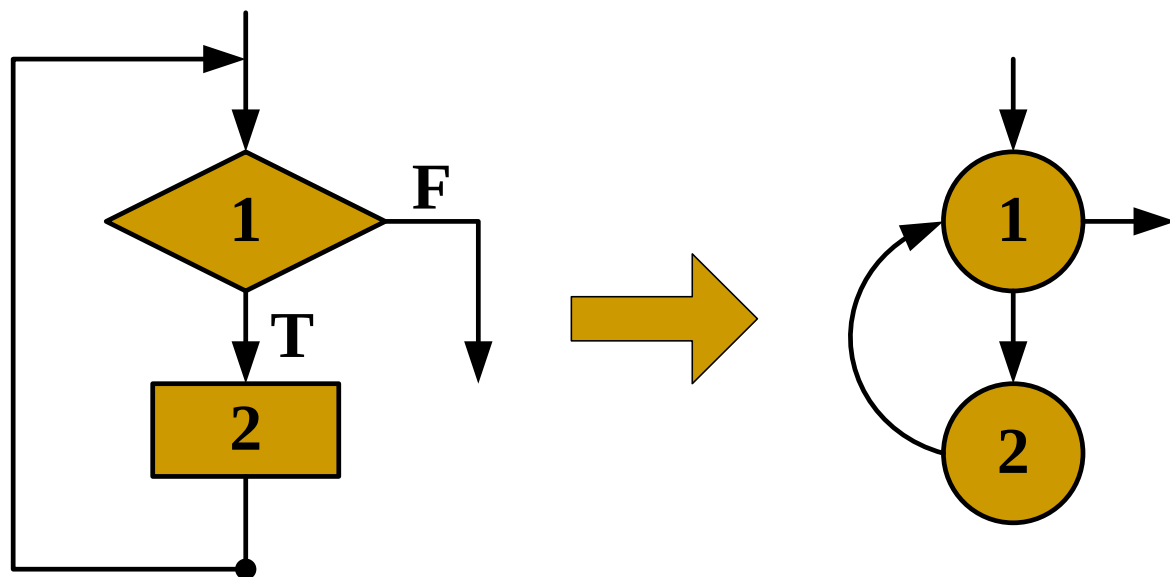
- 任何方法表示的过程设计结果，都可以翻译成流图。
- 对于顺序结构，一个顺序处理序列和下一个选择或循环的开始语句，可以映射成流图中的一个结点。

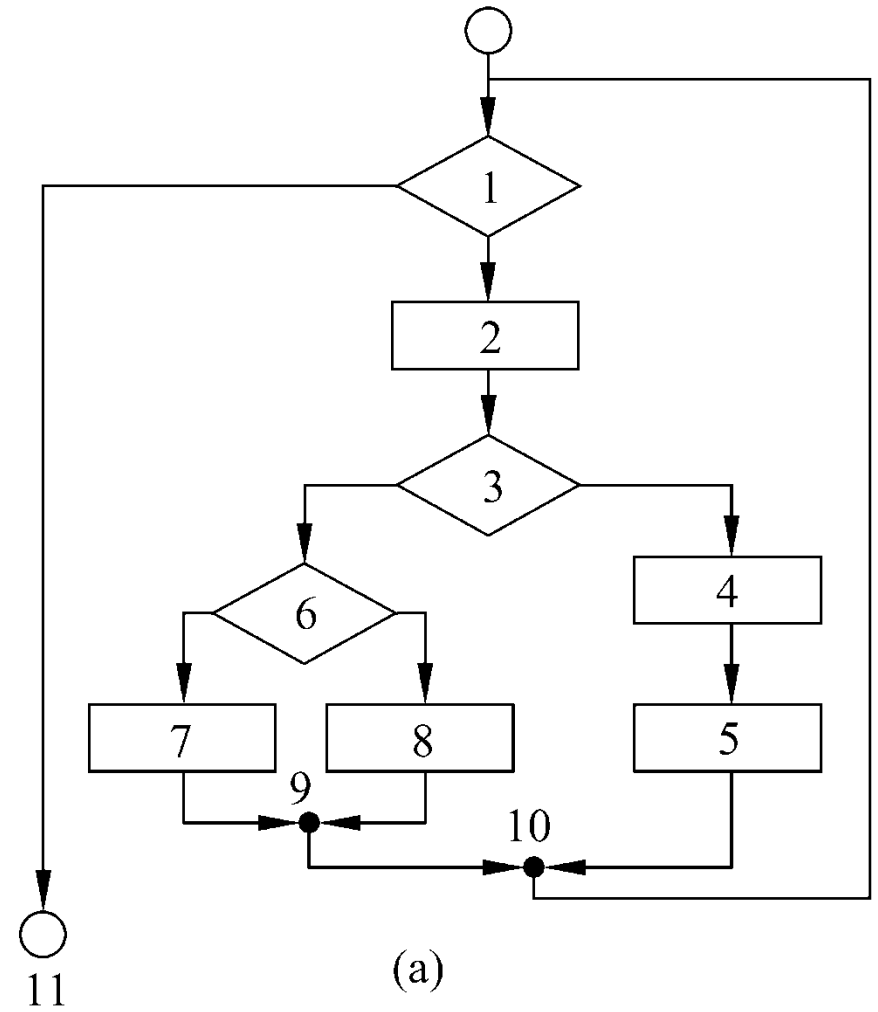
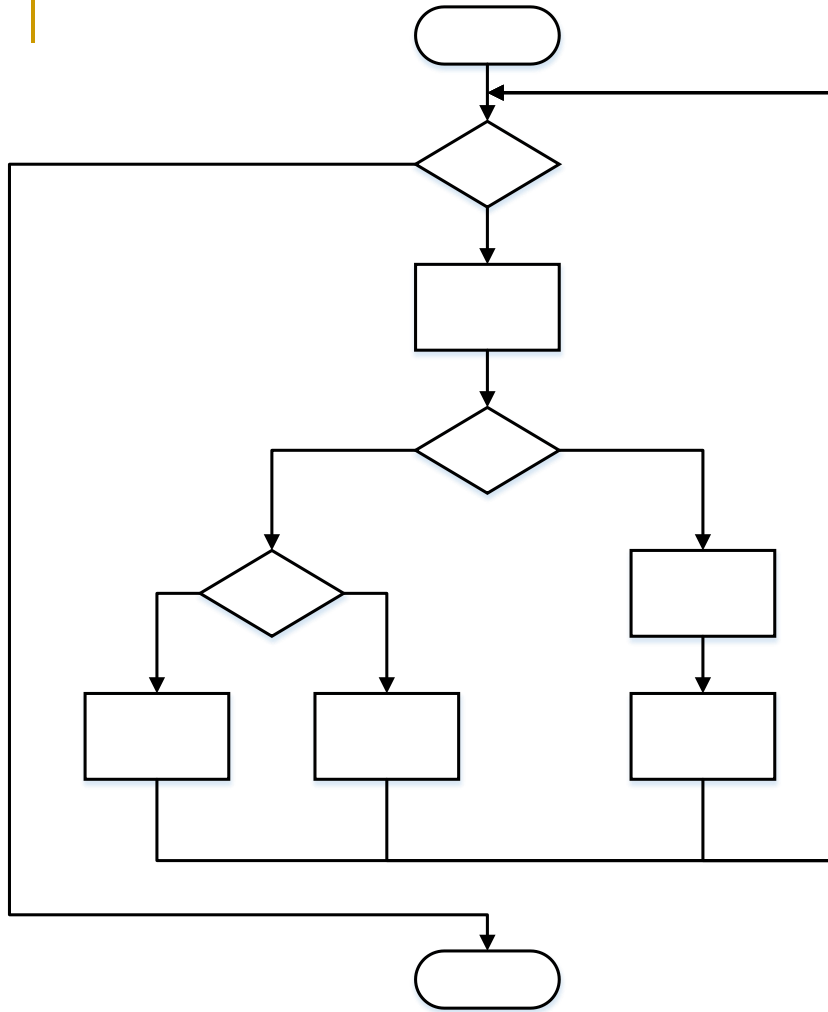


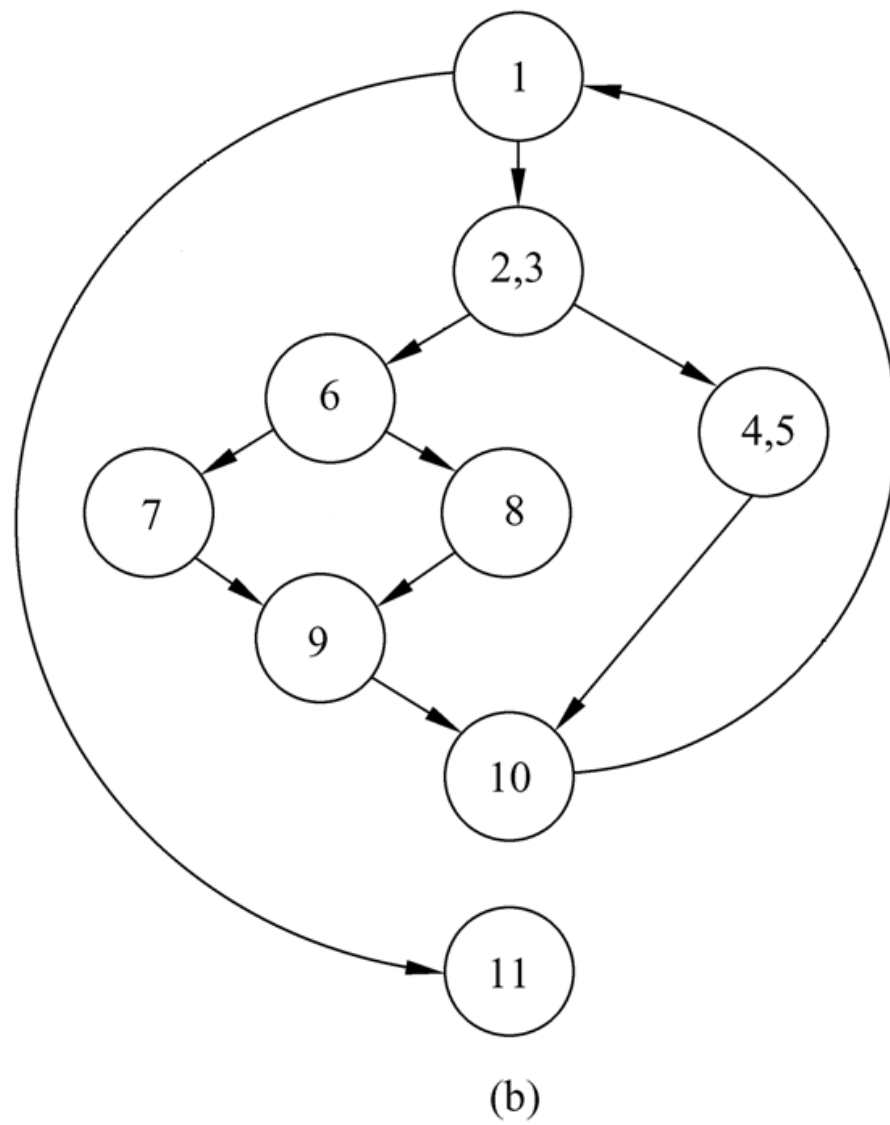
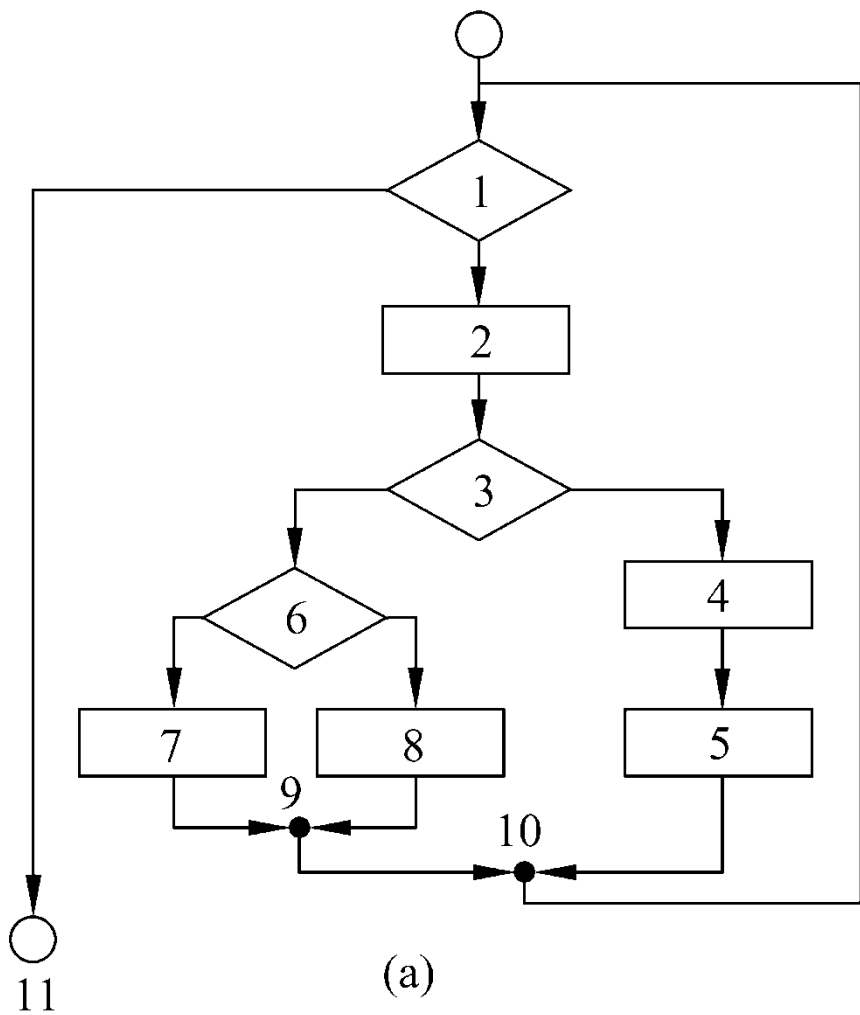
- 对于选择结构，开始语句映射成一个结点；两条分支至少各映射成一个结点；结束映射成一个结点。



- 对于循环结构，循环条件和循环体语句各映射成一个结点。





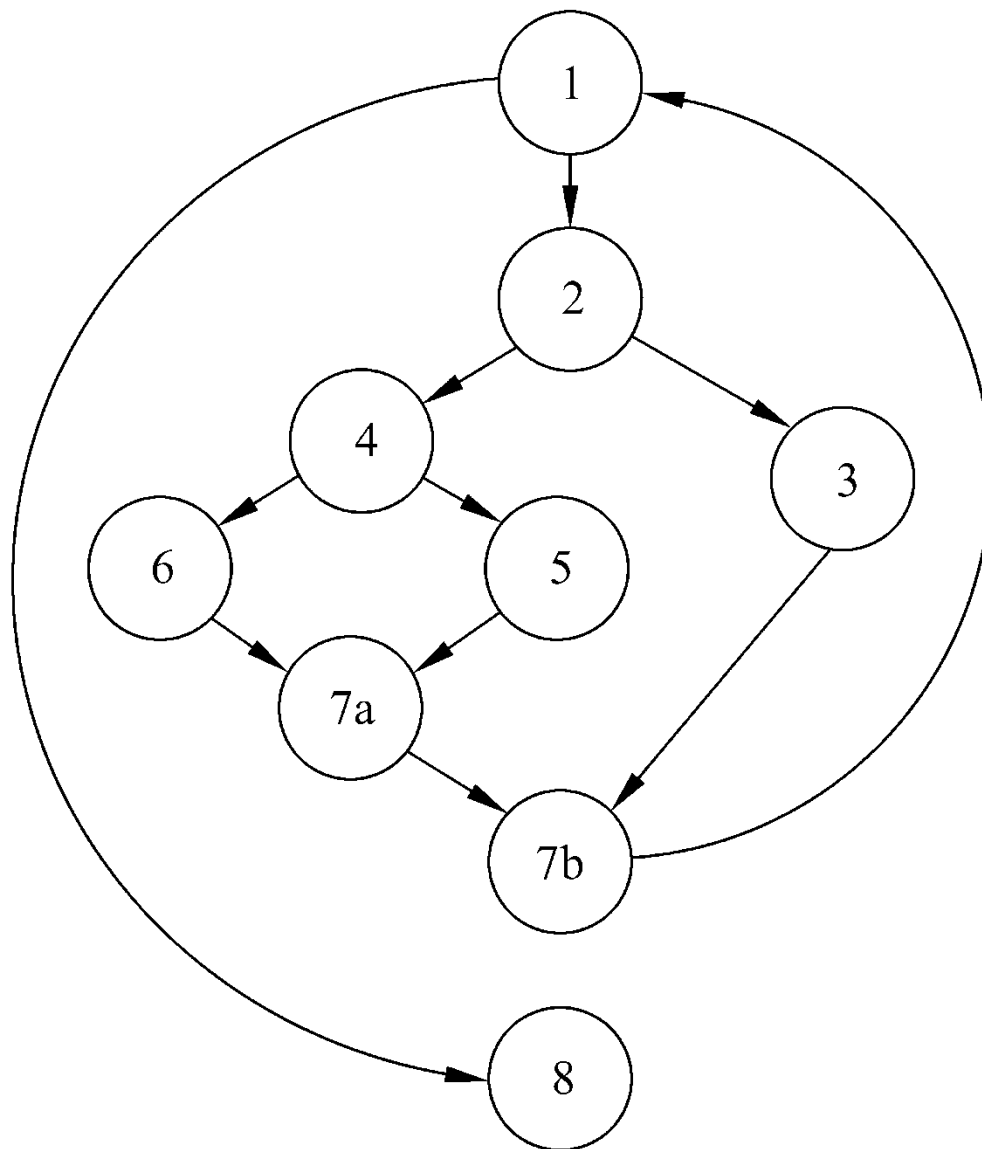


PDL

procedure:sort

- do while records remain
 - read record;
 - if record field 1=0
 - then process record;
store in buffer;
increment counter;
 - elseif record field 2=0
 - then reset counter;
 - else process record;
store in file;
 - endif
 - endif
- enddo
- end

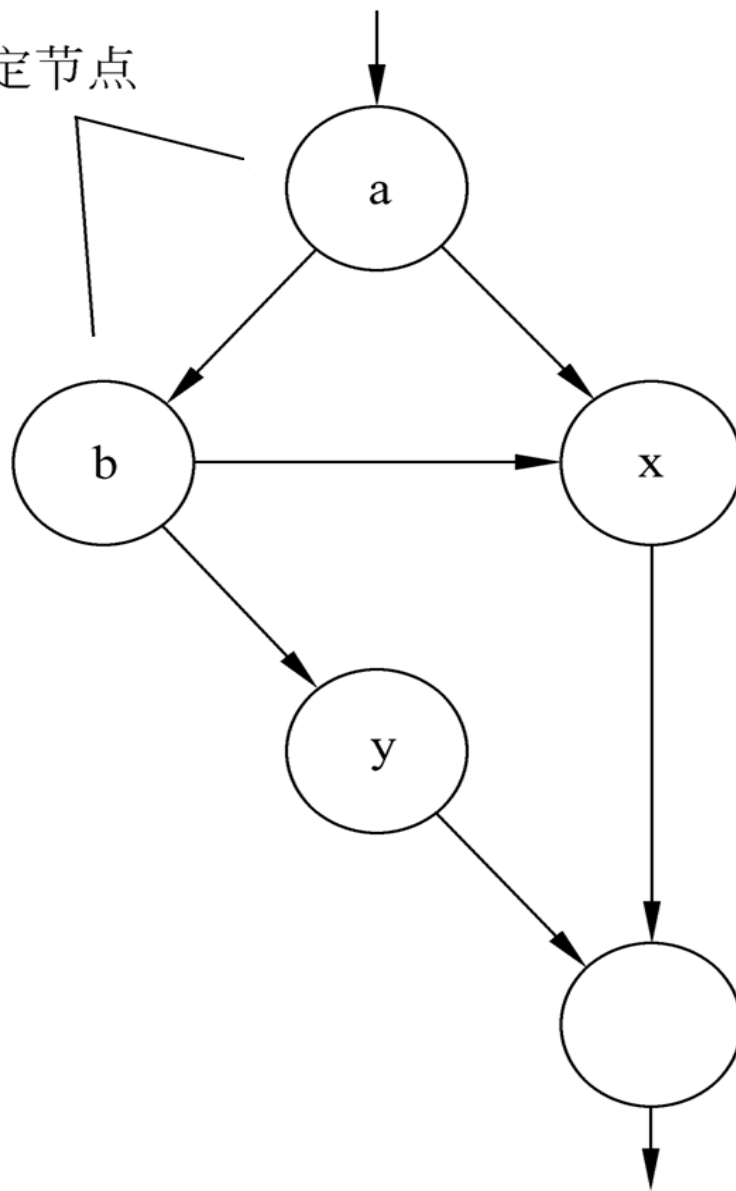
流图



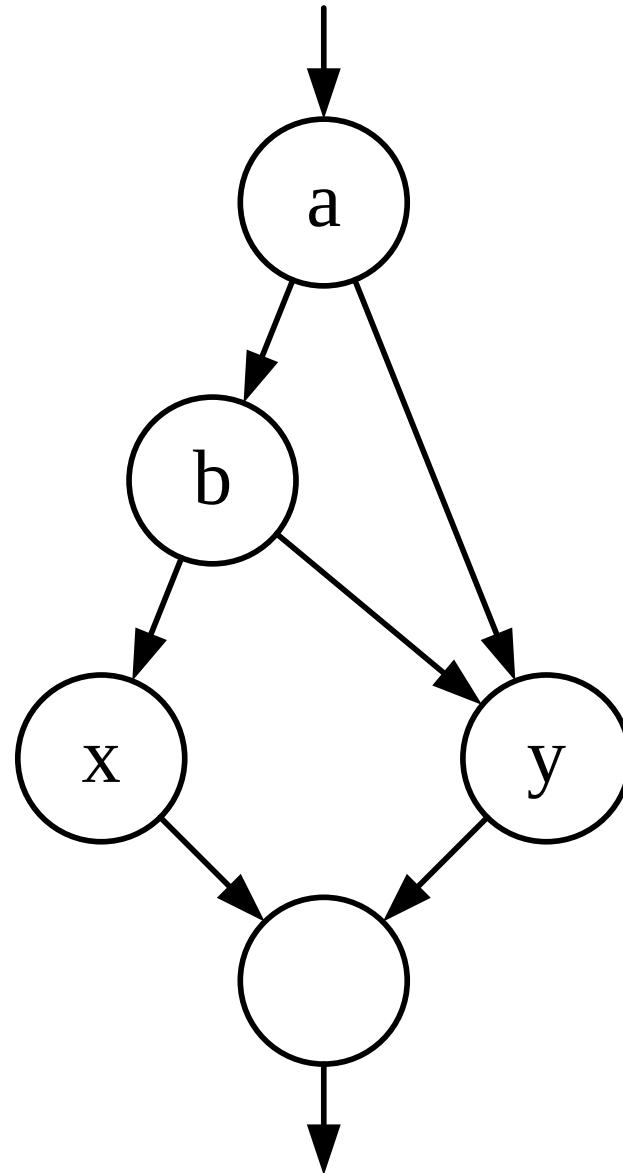
- 当过程设计中包含复合条件时，应该把复合条件分解为若干个简单条件，每个简单条件对应流图中一个结点。
- 所谓复合条件，就是在条件中包含了一个或多个布尔运算符(逻辑**OR**、**AND**)。

判定节点

⋮
IF a OR b
 then procedure x
 else procedure y
ENDIF
⋮



⋮
IF a AND b
 then procedure x
 else procedure y
ENDIF
⋮



2.计算环形复杂度的方法

- 环形复杂度定量度量程序的逻辑复杂度。有了描绘程序控制流的流图之后，可以用下述3种方法中的任何一种来计算环形复杂度 $V(G)$ 。

- $V(G)=$ 流图中的区域数

- $V(G)=E-N+2$

其中 E 是流图中的边数， N 是结点数

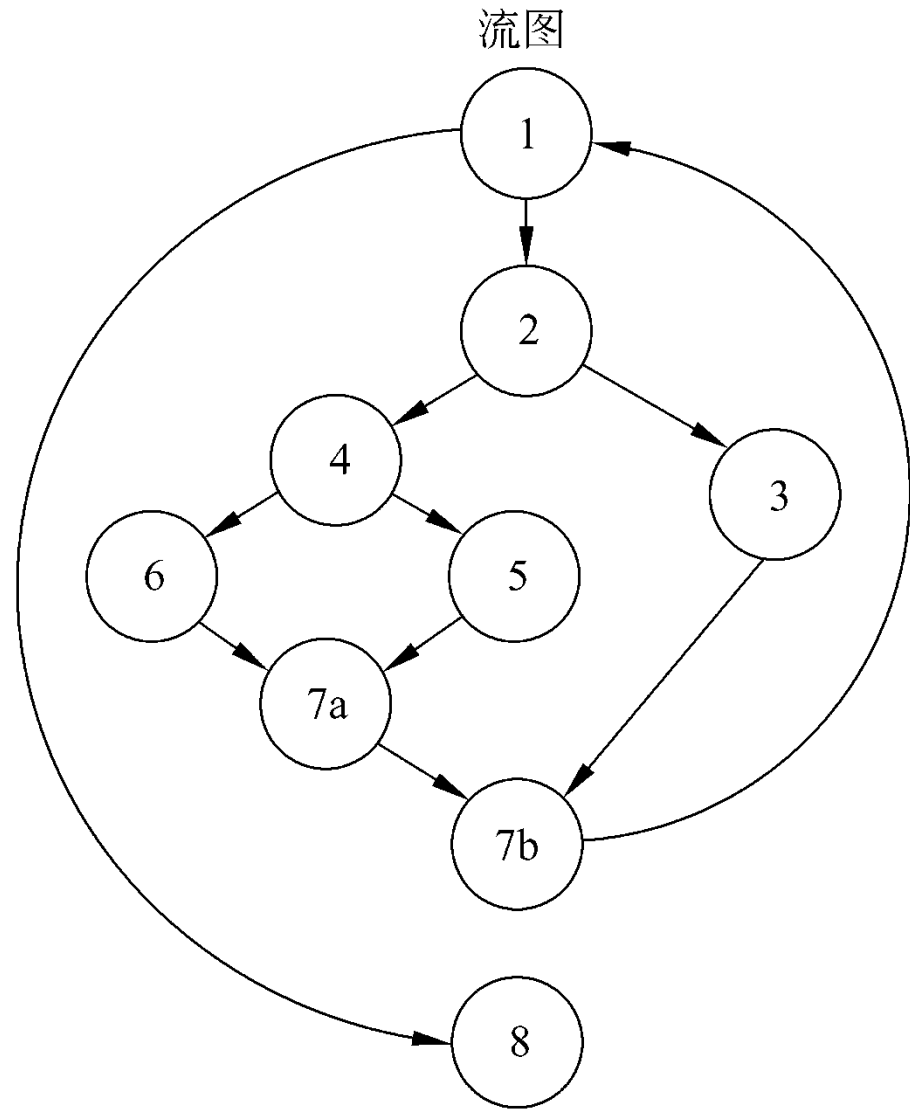
- $V(G)=P+1$

其中 P 是流图中判定结点的数目

■ $V(G) = \text{区域数}$
 $= 4$

■ $V(G) = E - N + 2$
 $= 11 - 9 + 2 = 4$

■ $V(G) = P + 1$
 $= 3 + 1 = 4$



3. 环形复杂度的用途

- 定量度量程序内分支数或循环个数，即程序结构的复杂程度；
- 定量度量测试难度；
- 能对软件最终的可靠性给出某种预测。
- 实践表明，模块规模以 $V(G) \leq 10$ 为宜。

2.4 面向对象详细设计

- 面向对象详细设计概念
- 面向对象详细设计步骤
- 面向对象详细设计工具（类图和顺序图）
- 类之间关系的详细设计
- 类的详细设计
- 面向对象详细设计例子

2.4 .1 面向对象详细设计概念

- 面向对象的详细设计从概要设计的对象和类开始，同时对它们进行完善和修改，以便包含更多的信息项，对概要设计进行扩展描述。
- 面向对象的详细设计主要对模块（构件）中的每个类进行详细设计，包括所有属性和与其实实现相关的操作。应该对每个属性的数据结构进行详细的说明，还需要说明实现与操作相关的处理逻辑的算法细节，说明实现接口所需机制的设计。
- 统一建模语言(Unified Modeling Language, UML)是由一整套图表组成的，为面向对象系统的产品进行说明、可视化和编制文档的一种标准语言。

2.4.2 面向对象详细设计步骤

- A. **类之间关系的详细设计**。可以使用UML类图、顺序图等描述类之间的关系。
- B. **类的详细设计**。对概要设计的类进行完善和修改，包含更多的信息项，例如：
 - 非功能性需求，如性能、输入和输出的约束
 - 用户界面的详细描述
 - 属性详细信息（数据结构详细信息）
 - 接口详细描述，包括类的继承接口和公共方法描述
 - 方法（操作）的详细实现描述，包括方法的实现流程和实现算法描述

2.4.3 面向对象详细设计工具

- 主要使用UML类图、顺序图（时序图）、通信图（协作图）、状态图等工具
- 类图主要描述模块的静态结构
- 顺序图、通信图、状态图主要描述模块的动态行为
- 也可使用面向过程的传统工具伪代码、程序流程图、判定表等对类中的方法流程进行描述。

10款好用的UML绘图软件

- boardmix: 无限白板画布, AIGC生成图表, 多功能的团队协作, 适用于多种办公绘图场景
- Microsoft Visio: Office 365集成, 丰富的模板库, 易用的协作功能
- Edraw Max: 丰富的模板库, Edraw AI支持
- Gleek.io: 使用键盘创建UML图, 简单而实用, 支持实时协作
- ClickUp: 多功能项目管理工具, 多视图, 实时协作, 移动支持
- Astah: 专业建模工具, 易用的UML功能
- Creately: 全面的图表和设计功能, 强调实时团队协作
- SmartDraw: 智能格式引擎, 众多模板, 协作工具
- Draw.io: 开源、安全, 高级绘图功能, 实时协作
- Moqups: 云端设计工具, 高级编辑功能, 支持交互原型制作

2.4.3.1 类图

- **类**：具有相同特性和行为的对象的抽象就是类，类的具体化就是对象。
 - 类具有属性，它是对象的状态的抽象，用数据结构来描述类的属性。
 - 类具有操作，它是对象的行为的抽象，用操作名和实现该操作的方法来描述。
- **消息**：对象之间进行通信的结构叫做消息。在对象的操作中，当一个消息发送给某个对象时，消息包含接收对象去执行某种操作的信息。

2.4.3.1 类图

■ 类之间的关系:

- 泛化(继承)
- 实现
- 关联
- 聚合
- 组合
- 依赖

2.4.3.1 类图

泛化(继承)

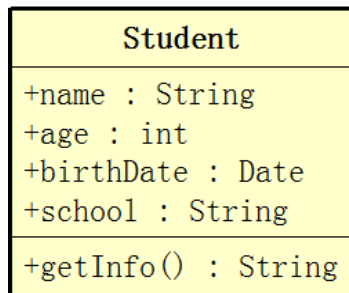
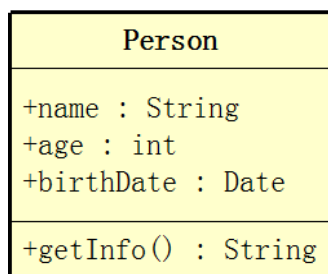
- **【泛化关系】**：是一种继承关系,它指定了子类如何继承父类的所有特征和行为。例如：老虎是动物的一种.
- **【箭头指向】**：带三角箭头的实线，箭头指向父类



2.4.3.1 类图

为什么要有继承？

- 1) 为描述和处理个人信息，定义类Person：
- 2) 为描述和处理学生信息，定义类Student：
- 3) 通过继承，简化Student类的定义

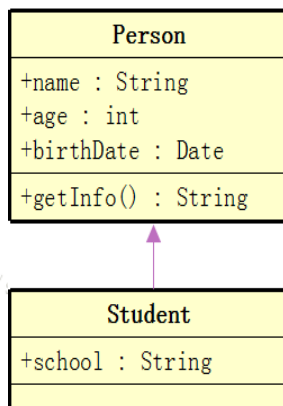


```
public class Person {  
    public String name;  
    public int age;  
    public Date birthDate;  
    public String getInfo()  
    {...}  
}
```

<https://blog.csdn.net/>

```
public class Student {  
    public String name;  
    public int age;  
    public Date birthDate;  
    public String school;  
    public String getInfo()  
    {...}  
}
```

<https://blog.csdn.net/shizhudufou>



```
public class Person {  
    public String name;  
    public int age;  
    public Date birthDate;  
    public String getInfo() {...}  
}
```

```
public class Student extends Person{  
    public String school;  
}
```

//Student类继承了父类Person的所有属性和方法，并增加了一个属性school。Person中的属性和方法，Student都可以利用。

2.4.3.1 类图

实现

- 【实现关系】：是一种类与接口的关系，表示类是接口所有特征和行为的实现
- 【箭头指向】：带三角箭头的虚线，箭头指向接口

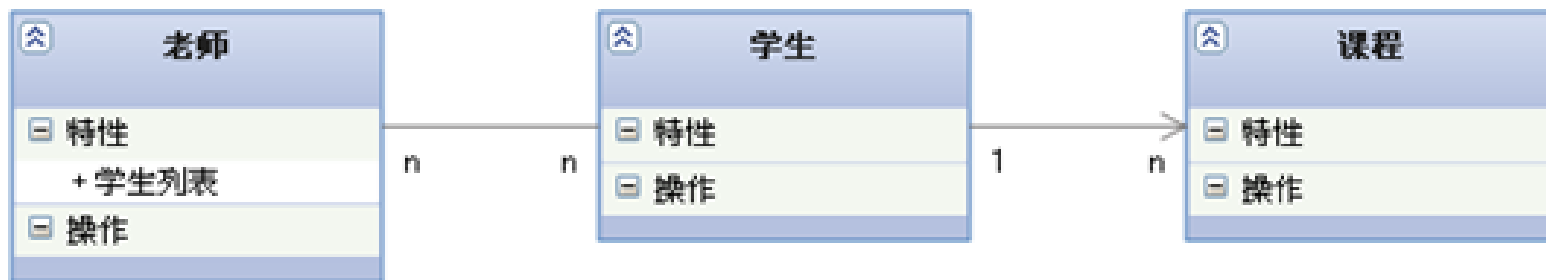


2.4.3.1 类图

关联

- **【关联关系】**：是一种结构化的关系，用于表示一类对象与另一类对象之间有联系；如：老师与学生，订单与客户
- 关联可以是双向的，也可以是单向的。双向的关联可以有两个箭头或者没有箭头，单向的关联有一个箭头。
- **【代码体现】**：成员变量

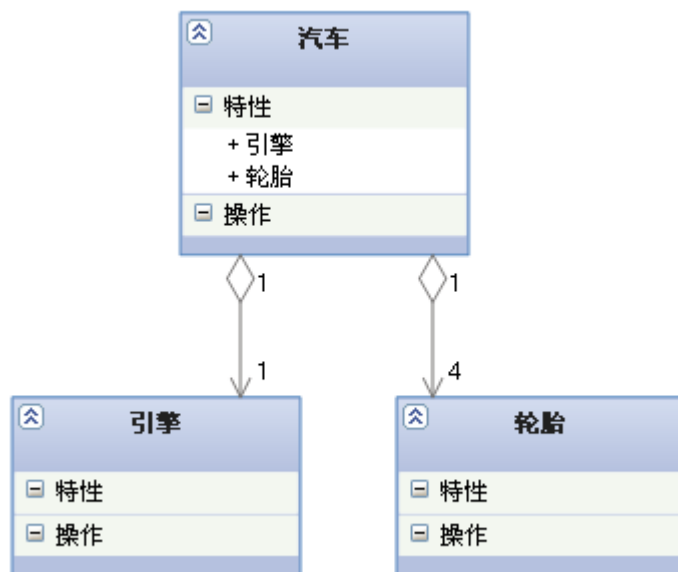
- **【箭头及指向】**：带普通箭头的实心线，指向被拥有者
老师与学生是双向关联，老师有多名学生，学生也可能有多名老师。但学生与某课程间的关系为单向关联，一名学生可能要上多门课程，课程是个抽象的东西他不拥有学生。



2.4.3.1 类图

聚合

- **【聚合关系】**：是整体与部分的关系.如车和轮胎是整体和部分的关系.
- 聚合关系是关联关系的一种，是强的关联关系；关联和聚合在语法上无法区分，必须考察具体的逻辑关系。
- **【代码体现】**：成员变量
- **【箭头及指向】**：带空心菱形的实心线，菱形指向整体



2.4.3.1 类图

组合

- **【组合关系】**：是整体与部分的关系，没有公司就不存在部门，组合关系是关联关系的一种，是比聚合关系还要强的关系，它要求普通的聚合关系中代表整体的对象负责代表部分的对象的生命周期
- **【代码体现】**：成员变量
- **【箭头及指向】**：带实心菱形的实线，菱形指向整体



2.4.3.1 类图

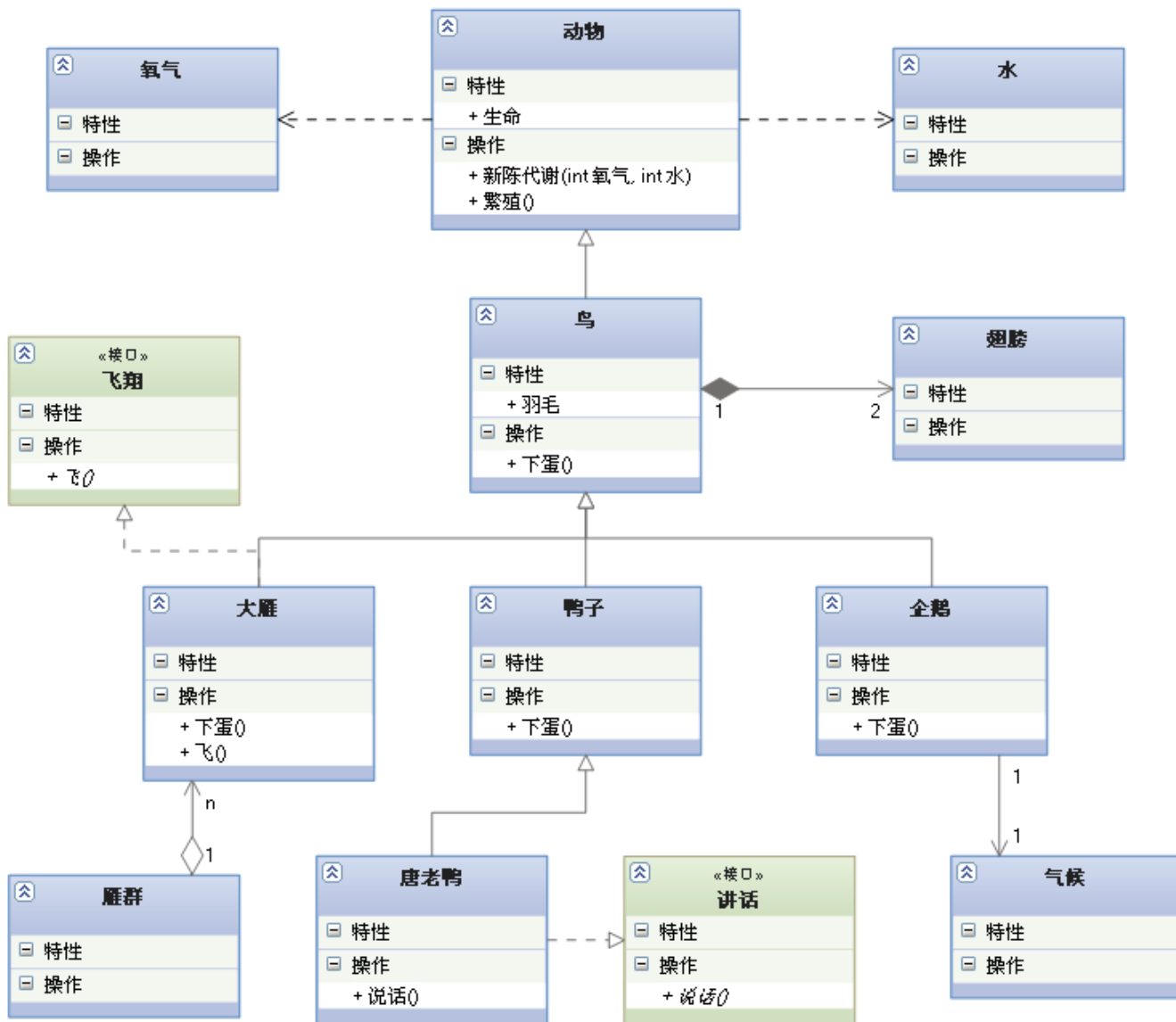
依赖

- **【依赖关系】**：是一种使用的关系，所以要尽量不使用双向的互相依赖。假设**A**类的变化引起**B**类的变化，说明**B**类依赖于**A**类。
- **【代码表现】**：局部变量、方法的参数或者对静态方法的调用
- **【箭头及指向】**：带箭头的虚线，指向被使用者



2.4.2.1 类图

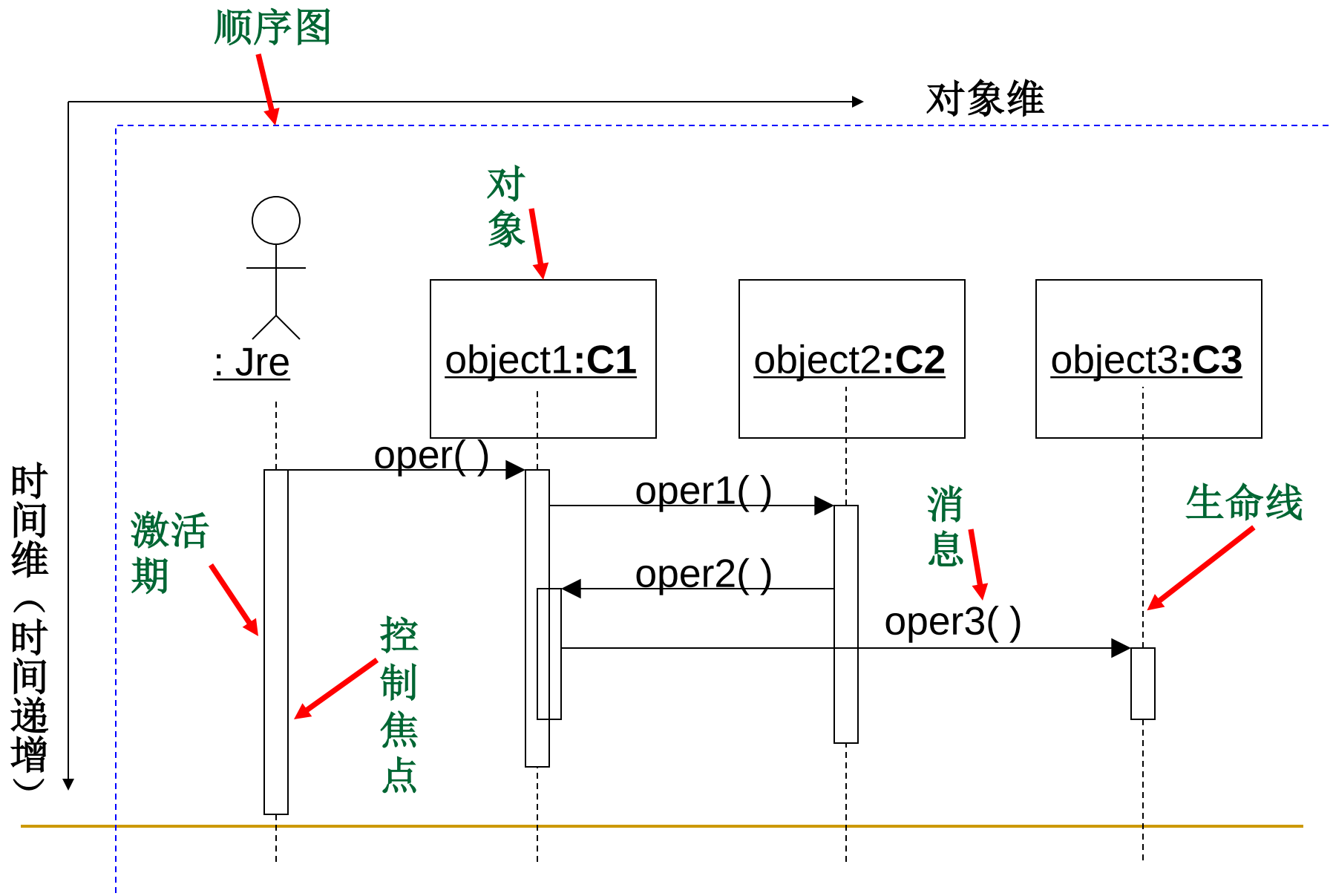
各种关系的强弱顺序：泛化= 实现 > 组合 > 聚合 > 关联 > 依赖



2.4.3.2 顺序图

- **顺序图——也称时序图**
- 描述对象之间按时间顺序进行的消息交互过程。
- 通常用来描述一个用例的行为，显示该用例中所涉及的对象和这些对象之间的消息传递情况
- 描述类的一个功能（用例）的实现过程

2.4.3.2 顺序图的概念



2.4.3.2 顺序图的概念

- **对象**
- 同类图中的对象，是类的实例
- **生命线**
- 从对象图标向下延伸的一条虚线，表示对象存在的生命期
- **控制焦点（激活期）**
- 对象执行一个动作的时间段
- **消息**
- 对象间的一次通信

- 消息的种类:

- 同步消息

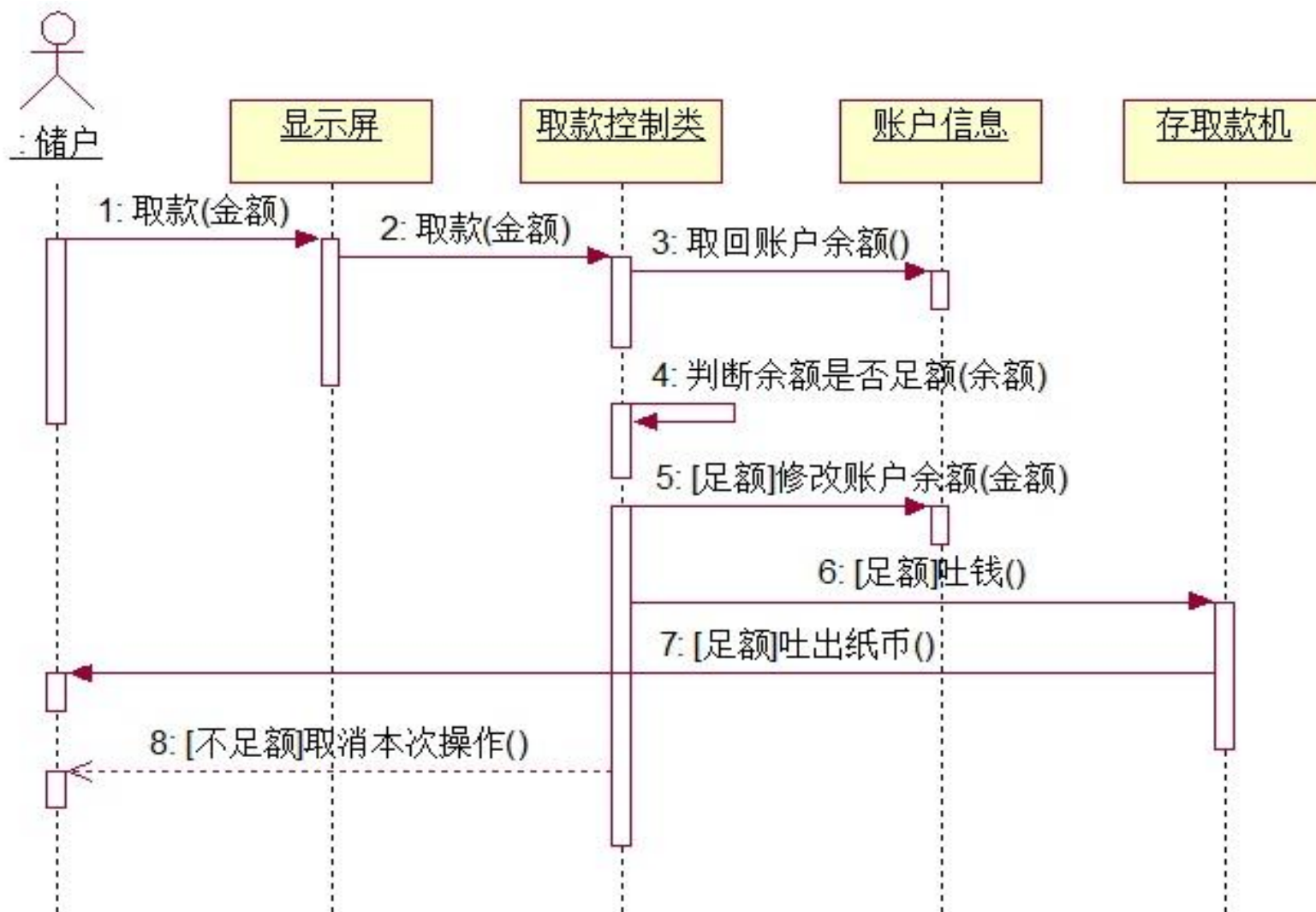
- 异步消息

- 返回消息

- 反身消息

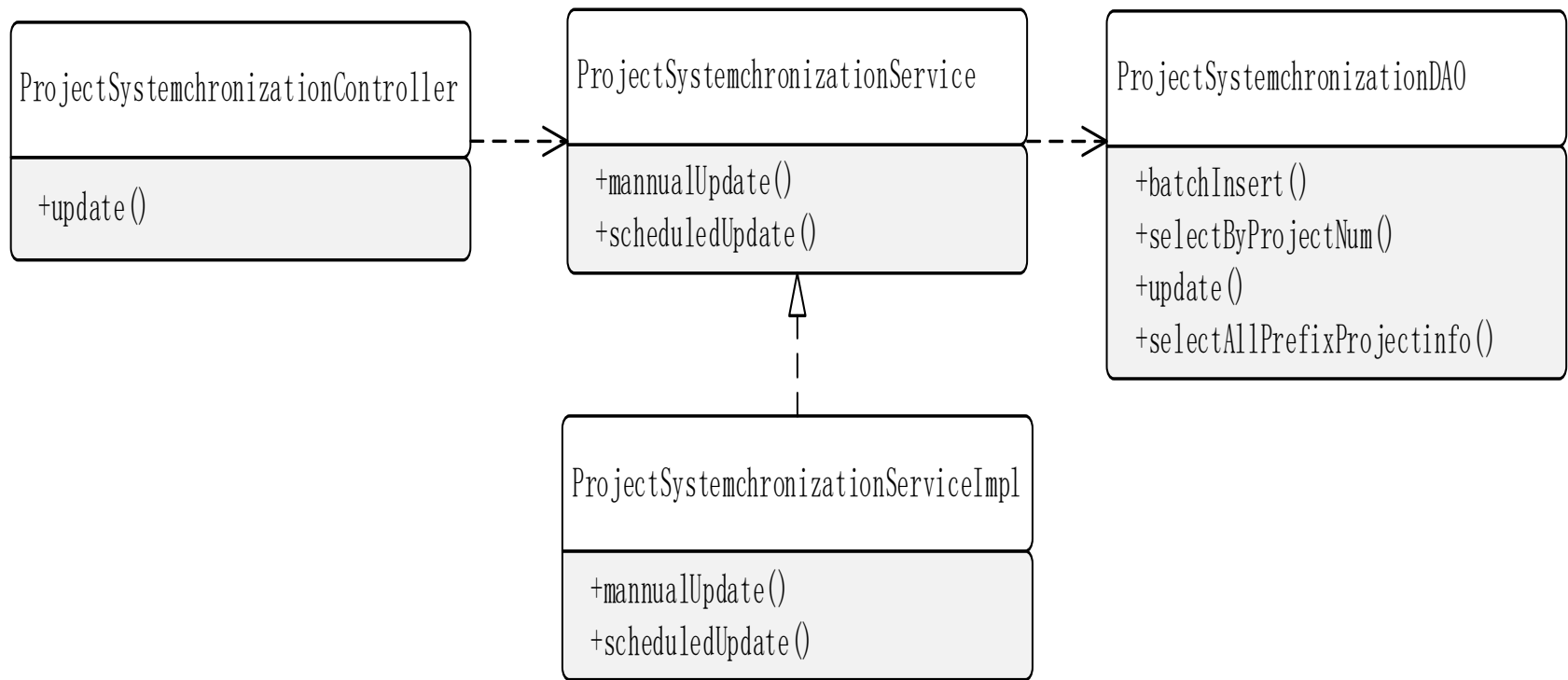
- 超时消息

- **例** 建设银行ATM机系统——取款
- 详细设计阶段的**顺序图**



2.4.4 类之间关系的详细设计

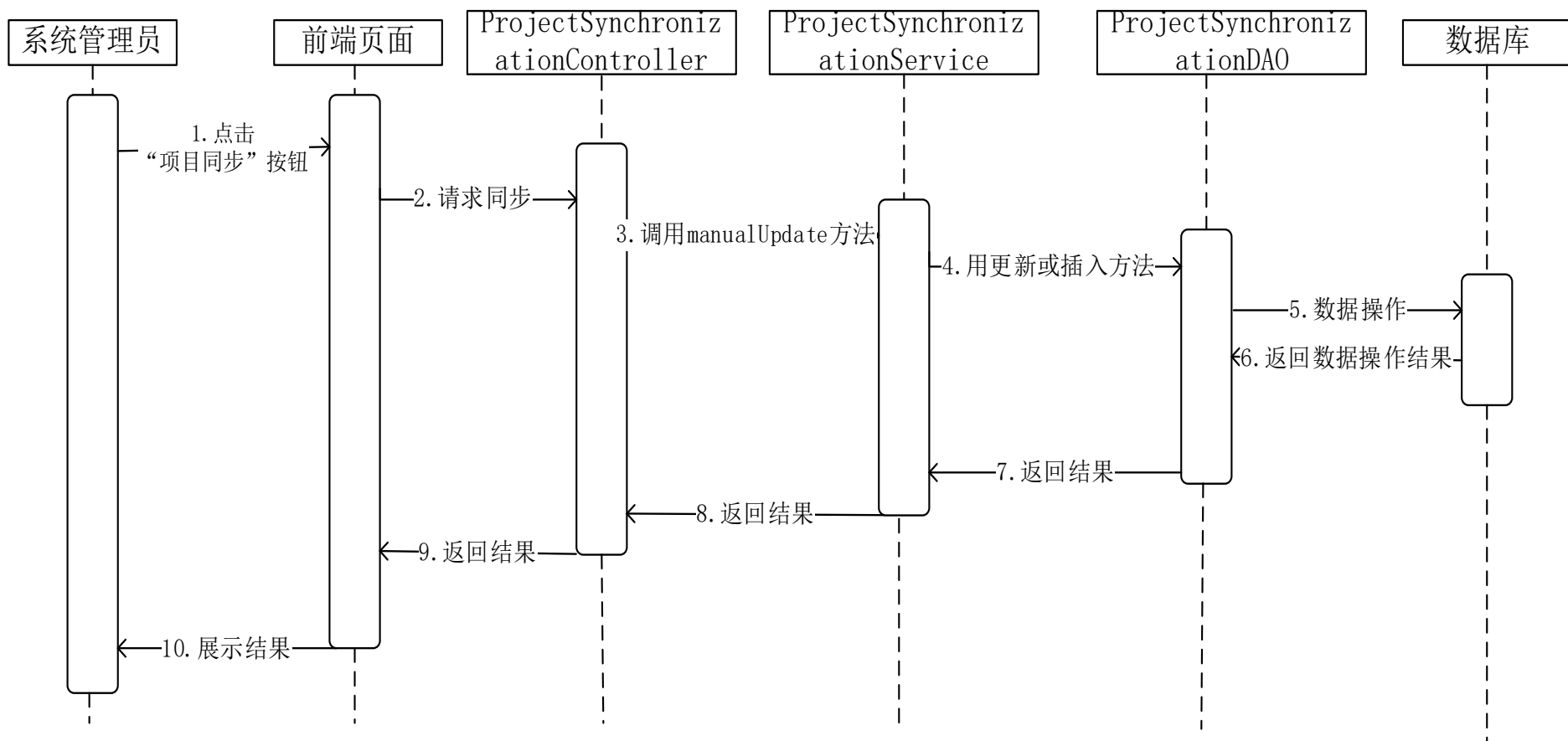
- 在详细设计阶段可以使用UML类图、顺序图等描述类之间的关系，例如某项目的架构设计采用的是分层架构，包括表现层、控制层、业务逻辑层和数据持久化层、其中一个功能模块(项目信息更新模块)的类图如下图所示，其功能需要通过分层架构实现。



项目信息更新模块类图

`ProjectSynchronizationController` 类是控制层类，`ProjectSynchronizationService` 接口和该接口的实现类 `ProjectSynchronizationServiceImpl` 是业务逻辑层类，`ProjectSynchronizationDAO` 类是数据持久层类。

■ 这个模块的具体实现逻辑如下顺序图所示。



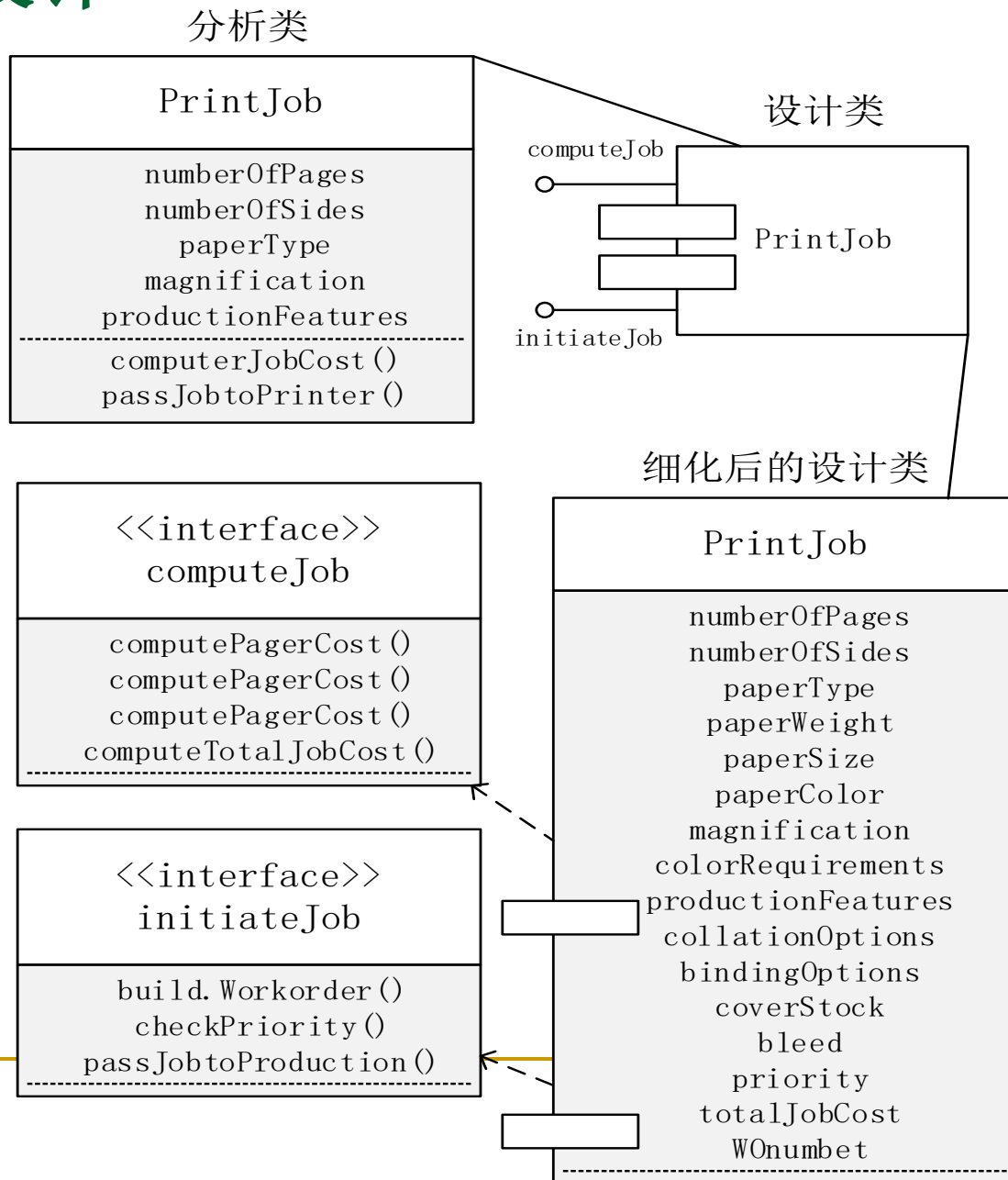
2.4.5 类的详细设计

- 详细设计阶段可能包含更多的类和对象，概要设计是高层描述数据的组织，而详细设计包含更多的数据结构信息。同时要说明每个对象的接口，规定每个操作的操作符号，以及对象的命名、每个对象的参数、方法的返回值。对象的接口是所有方法(操作)的名字的集合。对象实现描述是对对象内部的详细描述，包括：
 - 对象名字和引用类说明;
 - 私有数据结构、数据项和类型;
 - 每个方法的实现描述。可以通过伪代码、UML顺序图、程序流程图等对类中的方法进行描述

2.4.5 类的详细设计

(1) 类的详细描述

详细设计阶段需要对每类展开详细的描述，包括属性和方法的细化。



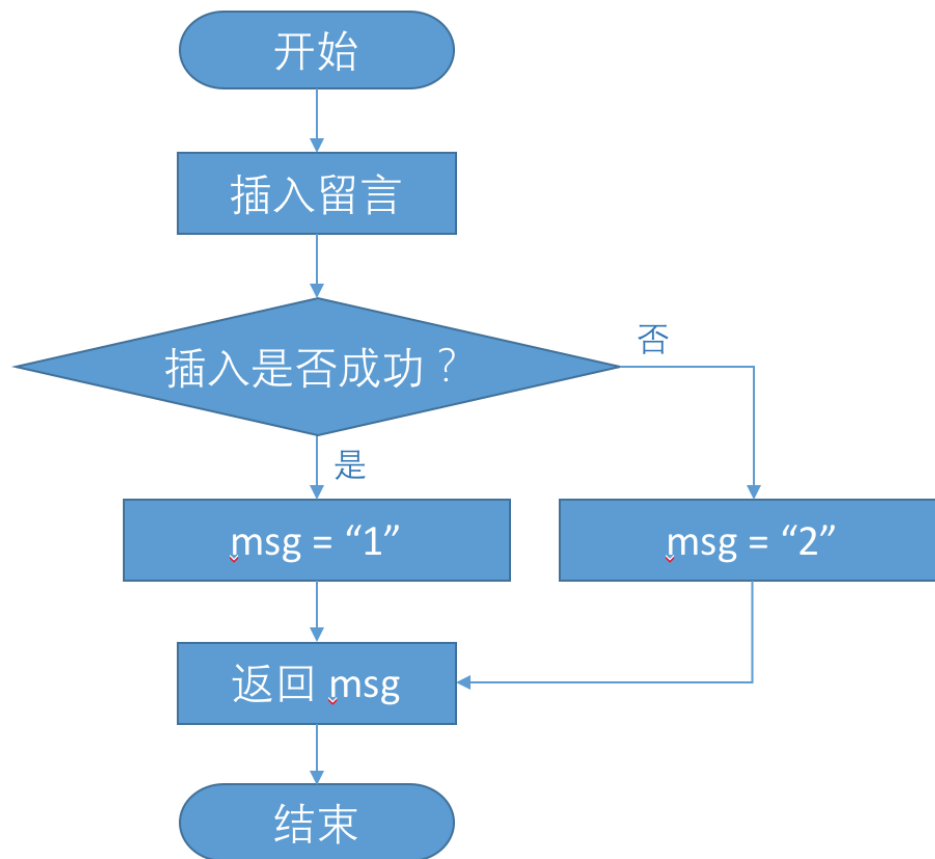
2.4.5 类的详细设计

(1) 类的详细描述

```
■ package com.buptsse.spm.action;
■ public class MessageAction extends ActionSupport{
■     /**
■         * 查询所有的留言内容
■         * @return
■         * @throws Exception
■         */
■     public String findMessageList();
■     /**
■         * 增加留言方法
■         * @return
■         * @throws Exception
■         */
■     public String insertMessage();
■     /**
■         * 删除留言方法
■         * @return
■         * @throws Exception
■         */
■     public String deleteMessage();
■ }
```

2.4.5 类的详细设计

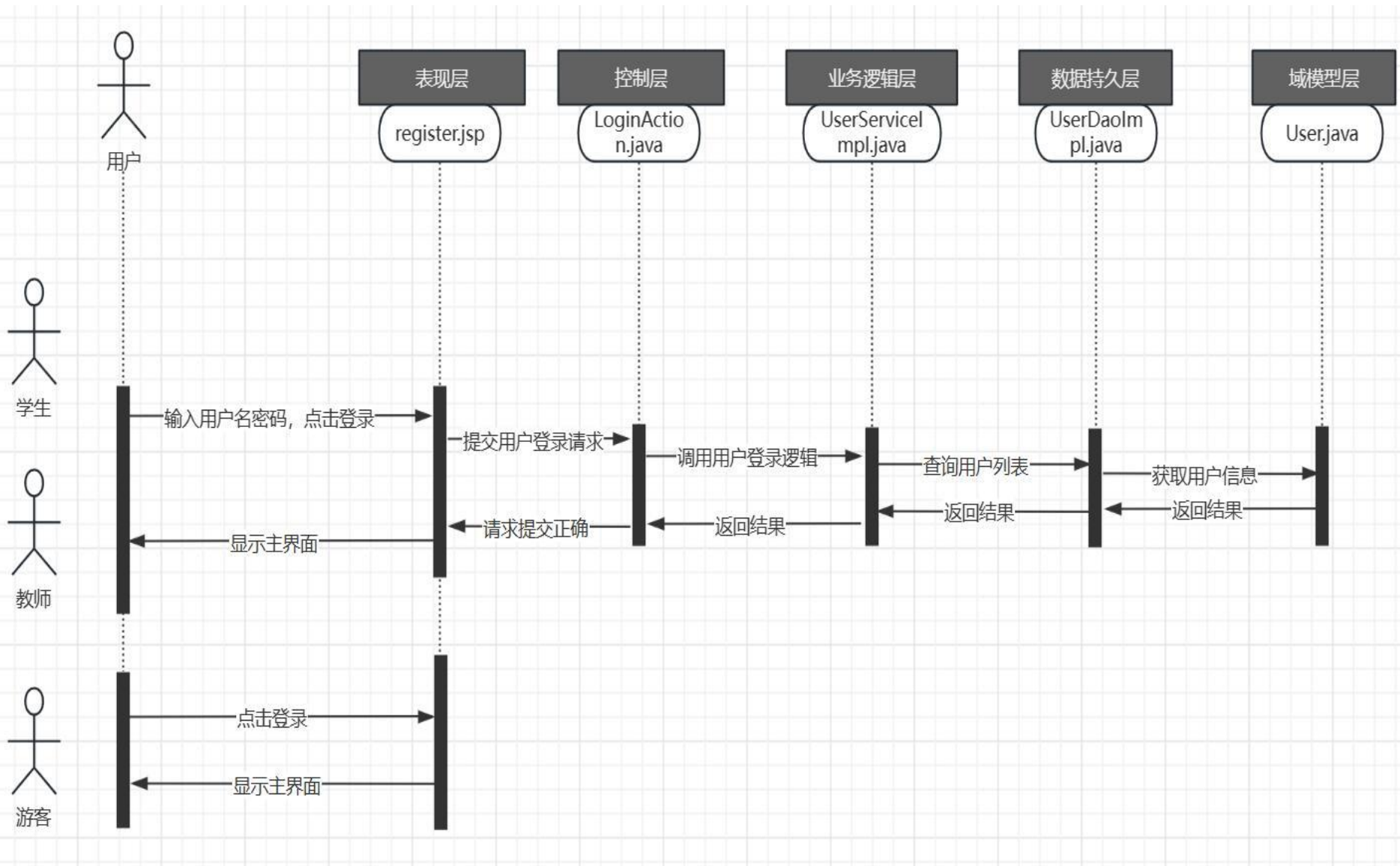
(2) 方法的详细设计



insertMessage()方法的流程图

2.4.6 面向对象详细设计例子

- 我们以某项目的登录模块为例说明其详细设计过程。该项目架构设计采用了多层结构，包括表现层、控制层、业务逻辑层、数据持久层和域模型层。登录模块系统内部的响应操作示意图如下图所示。



登录模块系统内部的响应操作示意图

2.4.6 面向对象详细设计例子

(1) 表现层

登录模块的表现层主要完成不同用户的登录功能。表现层对应的**JSP**页面列表如下表所示。

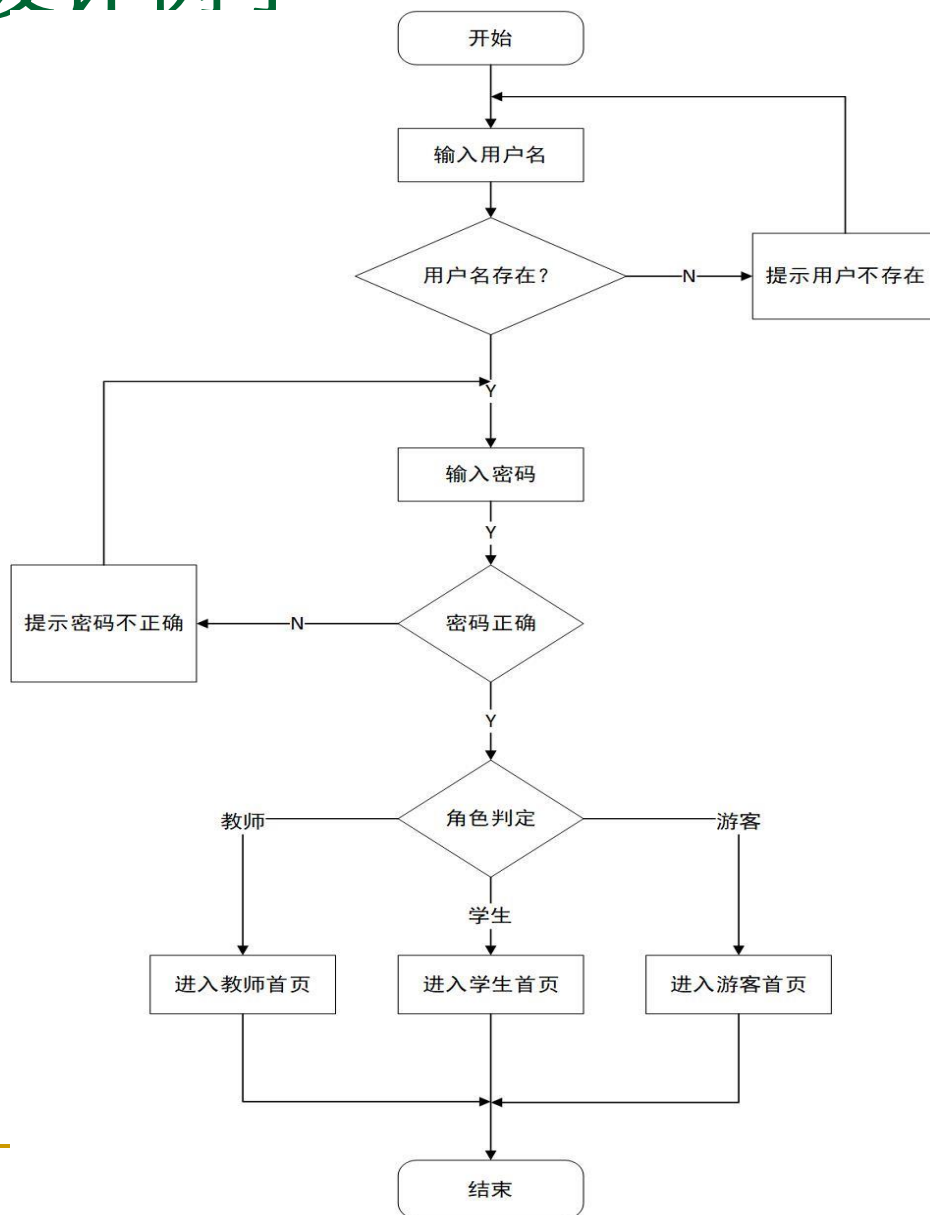
表5-5 登录模块表现层JSP列表

界面	JSP	功能描述
登录页面	login.jsp	用户(教师、学生、游客)登录功能，当登录出错给出提示。

2.4.6 面向对象详细设计例子

(1) 表现层

在login.jsp中编码逻辑的流程图。



2.4.6 面向对象详细设计例子

(2) 控制层

登录模块的控制层负责接收来自login.jsp的用户输入，同时调用登入模块的业务逻辑接口，将用户名与密码等用户关键信息传递到业务逻辑层进行判定。登录模块控制层列表如表5-6所示。

表5-6 登录模块控制层列表

事件	Action	转移说明	出口
登录	LoginAction.java	SUCCESS	mainFrame.jsp——登录成功，显示提示窗口
		ERROR	login.jsp——登录失败，显示提示窗口

2.4.6 面向对象详细设计例子

(2) 控制层

在控制层中LoginAction.java的主要属性与方法如下所示： 伪代码

```
package com.buptsse.spm.action;
/** @description 实现登录页面Action */
public class LoginAciton extends Actionsupport
{
    private User user;
    @Resource
    private luserService userService;
    /*实现各种属性的setter和getter方法*/
    /*实现登录模块的控制层方法*/
    public String login()
}
```

2.4.6 面向对象详细设计例子

(3) 业务逻辑层

登录模块的业务逻辑层主要完成对用户登录逻辑的判定，调用了公用的IUserService接口，同时实现该接口，登录模块业务逻辑层列表如表5-7所示。

表5-7 登录模块业务逻辑层列表

事件	Service	调用说明	出口
登录	IuserService.java UserServiceI mpl.java	调用 IuserDao	返回给 Action

(3) 业务逻辑层

IUserService.java接口主要方法如下:

```
package com.buptsse.spm.service;

public interface IUserService {
    /*登录检测*/
    public User login(String userName,
                      String password);
    /*查找用户*/
    public User findUser(String userName);
    /*删除用户*/
    public boolean deleteUser(String userName);
    /*增加用户*/
    public boolean addUser(User user);
}
```

UserServiceImpl.java实现了IUserService.java接口，主要实现的属性与方法如下

```
package com.buptsse.spm.service.imp;

@Transactional
@Service
public class UserServiceImpl implements IUserService {
    @Resource
    private IUserDao iUserDao;
    @Override
    public User login(String userName, String password);
    @Override
    public User findUser(String userName);
    @Override
    public boolean deleteUser(String userName);
    @Override
    public boolean addUser(User user);
}
```

2.4.6 面向对象详细设计例子

(4) 数据持久层

登录模块的数据持久层能对用户数据进行增删改查。登录模块数据持久层列表如表5-8所示。

表5-8 登录模块数据持久层列表

事件	DAO	调用数据模型	说明
登录	IUserDao. java UserDaoImpl. java	调用 user. java	对用户信息 进行增删改 查操作

（4）数据持久层

IUserDao.java接口是定义对用户信息进行增、删、改、查的接口

```
package com.buptsse.spm.dao;
import java.util.List;
/**
 * @description 用户信息持久层接口定义,
 * 包括用户的增加、保存、查询
 */
public interface IUserDao {
    public User findUser(User user);
    /*查找用户信息*/
    public boolean insertUser(User user);
    /* 插入用户信息*/
    public List<User> searchUser(List<Object>
                                choose);
    /*查找用户信息*/
    public boolean deleteUser(String username);
    /*删除用户信息*/
    public boolean addUser(User user);
    /*增加用户信息*/
}
```

UserDaoImpl.java是IUserDao.java接口的实现

```
package com.buptsse.spm.dao;
/**
 * @description 用户信息持久层接口实现,
 * 包括用户增加, 保存, 查询
 */
public class UserDaoImpl extends BaseDAOImpl
    implements IUserDao{
    public User findUser(User user);
    /*查找用户信息*/
    public boolean insertUser(User user);
    /*插入用户信息*/
    public List<User> searchUser(List<Object>
                                choose);
    /*查找用户信息*/
    public boolean deleteUser(String username);
    /*删除用户信息*/
    public boolean addUser(User user);
    /*增加用户信息*/
}
```

2.4.6 面向对象详细设计例子

(5) 域模型层

登录模块用到了域模型层中的User.java, User.java是一个公用域模型,在涉及用户信息查询等操作时,就会调用该模型,登录模块域模型层列表如表5-9所示。

表5-9 登录模块域模型层列表

域模型	描述
User. java	存储用户信息, 各种用户属性

(5) 域模型层

User.java主要属性与方法如下：

```
package com.buptsse.spm.domain;
import java.io.Serializable;
/**
 * @description User 表的信息记录
 */
public class User implements Serializable {
    private String id;          /*id 识别*/
    private String userName;    /*用户名*/
    private String password;    /*密码*/
    private String password1;   /*验证密码*/
    private String position;    /*身份信息*/
    private String userId;      /*用户 ID*/
    private int videoTime;      /*用户学习时间*/
    /*编写对上述各个属性的 getter 与 setter 方法*/
}
```


2.5 人机界面设计

- 人机界面设计是接口设计的重要组成部分。对于交互式系统来说，人机界面设计和数据设计、体系结构设计及过程设计一样重要。
- 人机界面的设计质量，直接影响用户对软件产品的评价，从而影响软件产品的竞争力和寿命，必须对人机界面设计给予足够重视。

2.5 人机界面设计

三条“黄金规则”：

- 置用户于控制之下。
- 减少用户记忆负担。
- 保持界面一致。

2.5.1 设计问题

设计人机界面过程中会遇到的4个问题：

- 系统响应时间
- 用户帮助设施
- 出错信息处理
- 命令交互

2.5.1 设计问题

1. 系统响应时间

- 系统响应时间指从用户完成某个控制动作，到软件给出预期的响应之间的这段时间。系统响应时间有两个重要属性：长度和易变性。
- **长度：**如果系统响应时间过长，用户就会感到紧张和沮丧；系统响应时间过短会迫使用户加快操作节奏，从而可能会犯错误。
- **易变性：**指系统响应时间相对于平均响应时间的偏差，即使系统响应时间较长，响应时间易变性低也有助于用户建立起稳定的工作节奏。

2.5.1 设计问题

2. 用户帮助设施

- 大多数现代软件都提供联机帮助设施，用户无须离开用户界面就能解决自己的问题。常见的帮助设施可分为集成的和附加的两类。
- **集成的帮助设施**设计在软件里面，它对用户工作内容是敏感的，用户可以从与刚刚完成的操作有关的主题中选择一个请求帮助。
- **附加的帮助设施**是在系统建成后再添加到软件中的，它实际上是一种查询能力有限的联机用户手册。
- **集成的帮助设施优于附加的帮助设施。**

2.5.1 设计问题

3. 出错信息处理

- 出错信息和警告信息，是出现问题时交互式系统给出的“坏消息”。

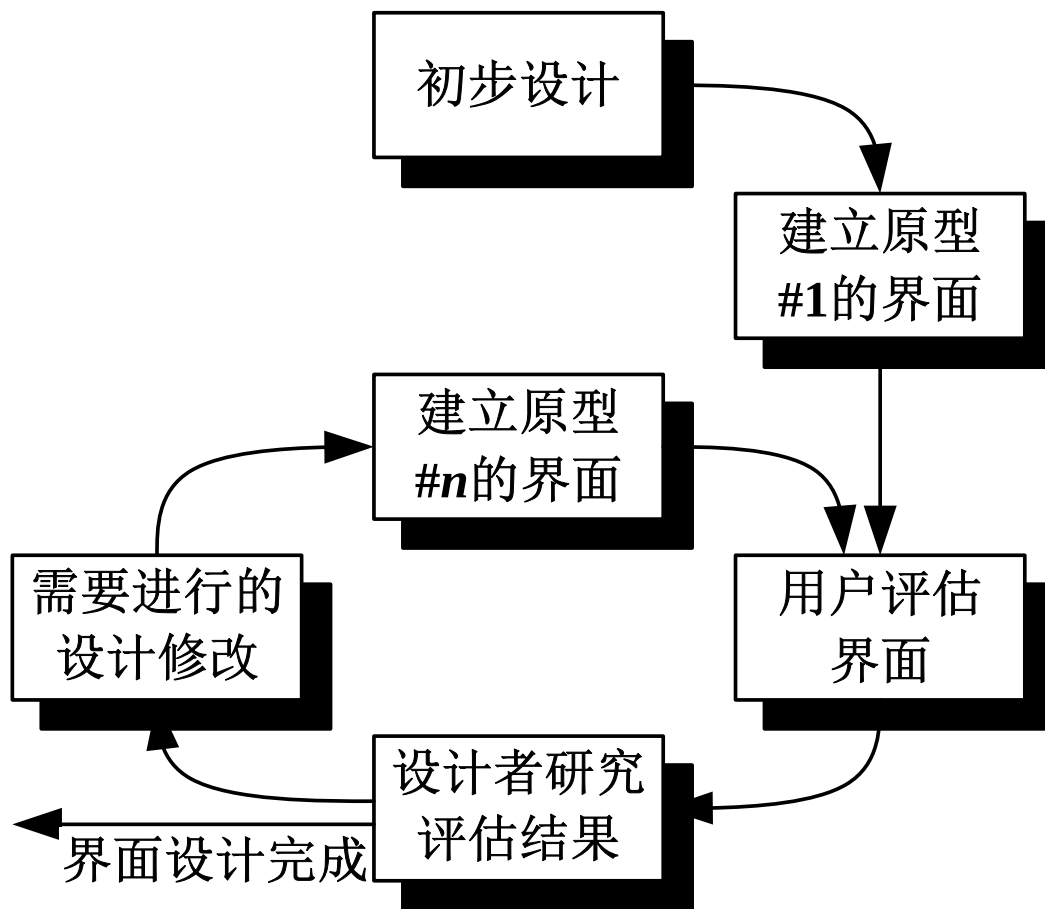
4. 命令交互

- 多数情况下，用户既可以从菜单中选择软件功能，也可以通过键盘命令序列调用软件功能。
- 在理想的情况下，所有应用软件都有一致的命令使用方法。

2.5.2 设计过程

- 用户界面设计是一个迭代的过程，也就是说，通常先创建设计模型，再用原型实现这个设计模型，并由用户试用和评估，然后根据用户意见进行修改。
- 为了支持上述迭代过程，各种用于界面设计和原型开发的软件工具应运而生。这些工具被称为用户界面工具箱或用户界面开发系统。
- 界面原型设计工具：Axure RP、墨刀、Microsoft Visio

2.5.2 设计过程



界面设计评估周期

2.5.3 人机界面设计指南

- 界面分类
 - 一般交互界面
 - 信息显示界面
 - 数据输入界面

一般交互界面

■ 基本设计指南

- 保持一致性；
- 提供有意义的反馈和帮助信息；
- 执行较大破坏性动作之前要求用户确认；
- 允许犯错误，允许中断或撤销绝大多数操作；
- 减少两次操作之间必须记忆的信息量；
- 按功能对动作分类，并据此设计屏幕布局；
- 用简单动词或短语作为命令名

一般交互界面

■ 交互界面的基本类型

- ① 菜单界面
- ② 对话
- ③ 其他

一般交互界面

■ ①菜单界面

- 按显示形象分类:

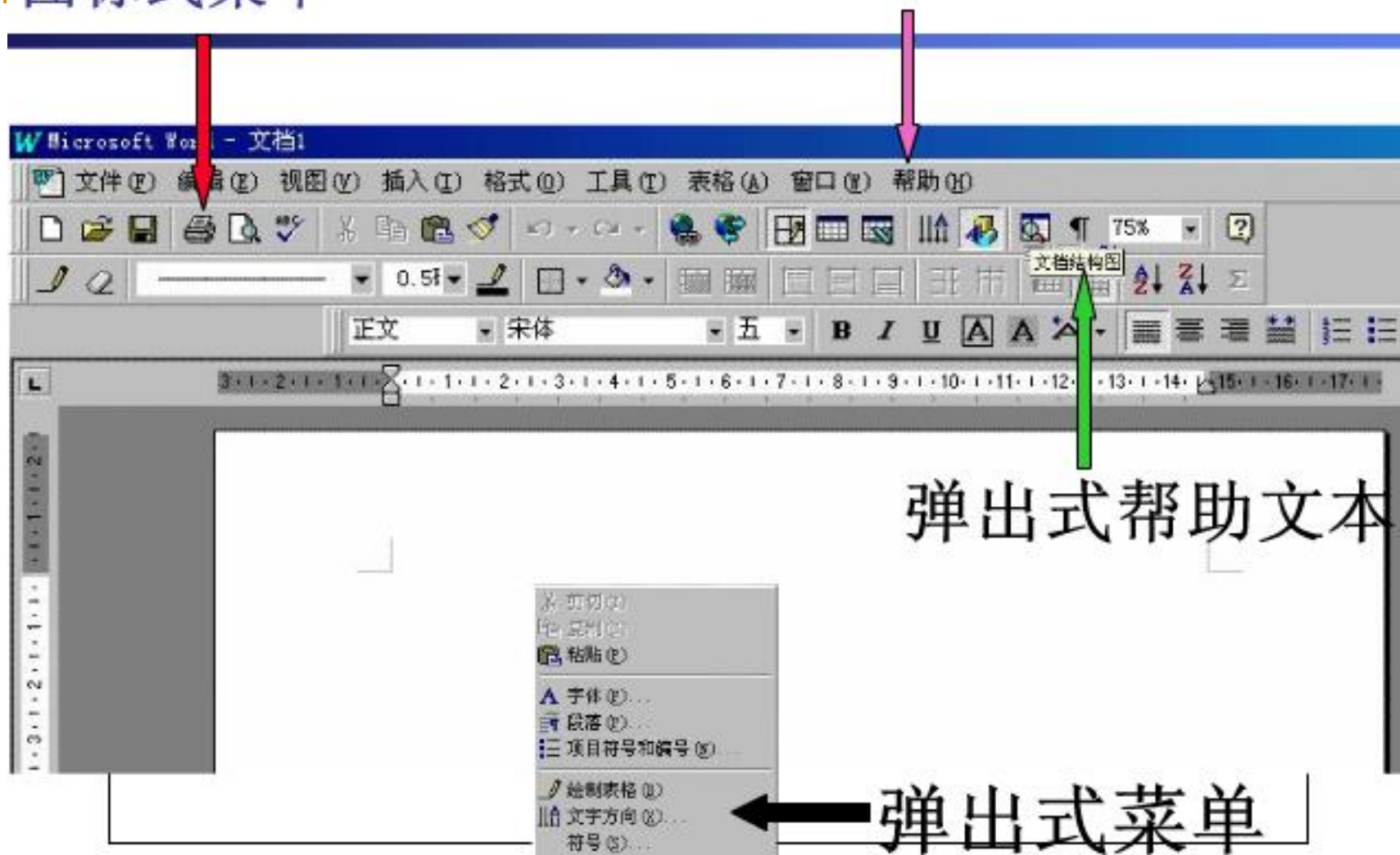
- 正文菜单、图标菜单、正文图标混合菜单。

- 按屏幕位置和操作风格分类

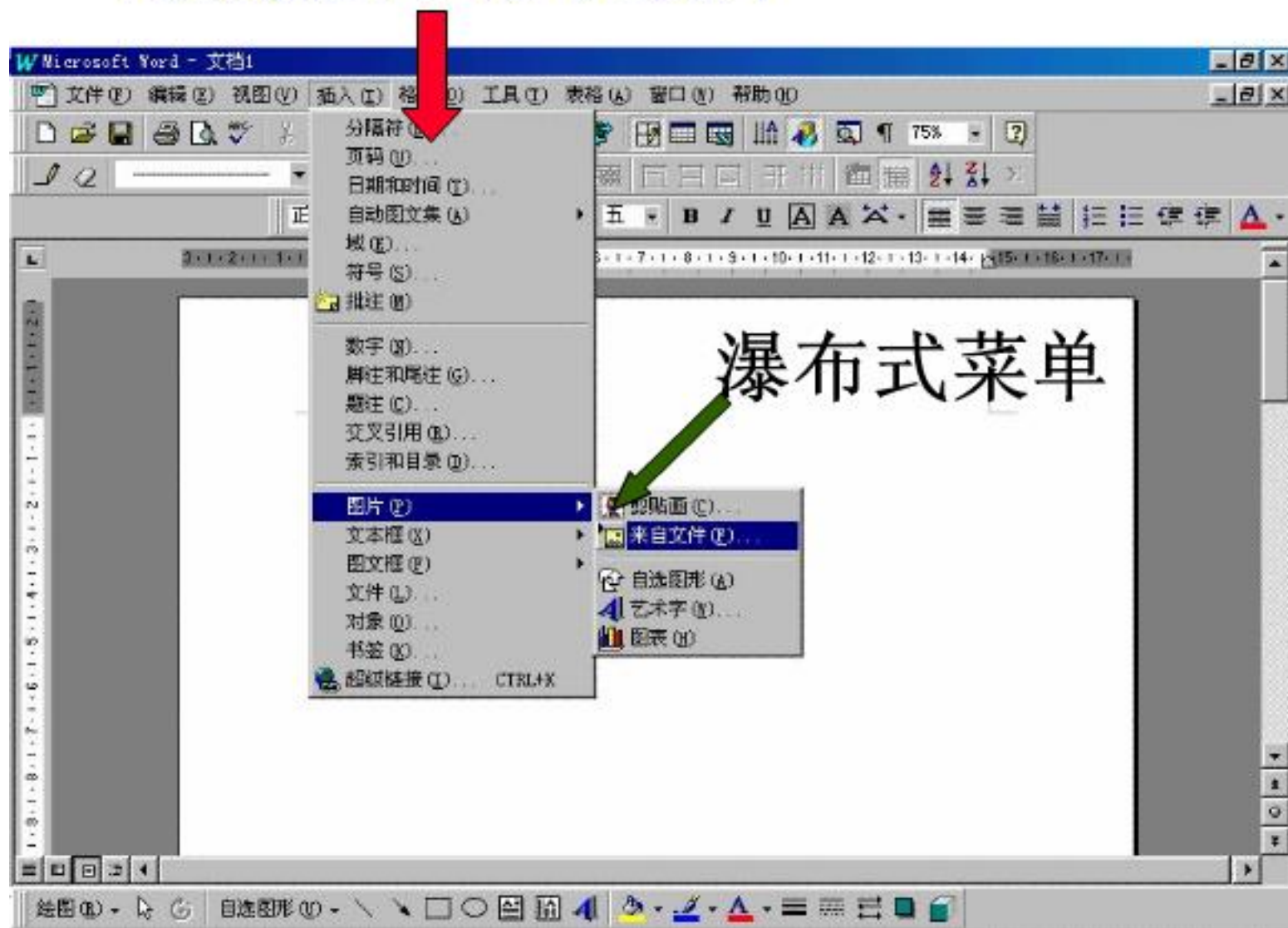
- 固定、浮动、下拉式、嵌入式

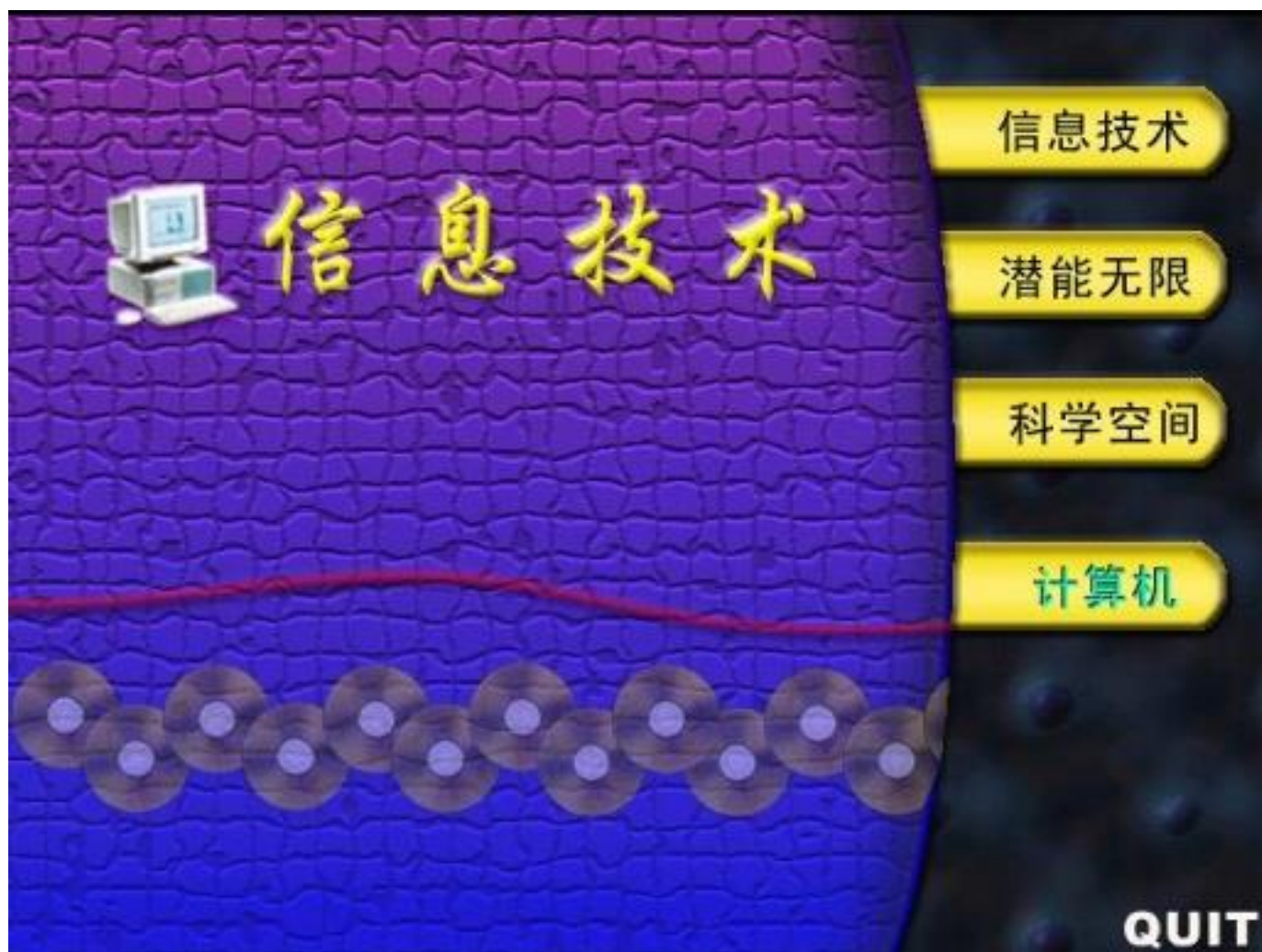
图标式菜单

菜单条



下拉式菜单、瀑布式菜单





一般交互界面

- ② 对话框
 - 对话形式
 - 必须回答式、无需回答式、警告式
- ③ 其他
 - 图像
 - 窗口

信息显示界面

■ 界面显示的基本元素

- ①数据、②字符、③图形、④图像

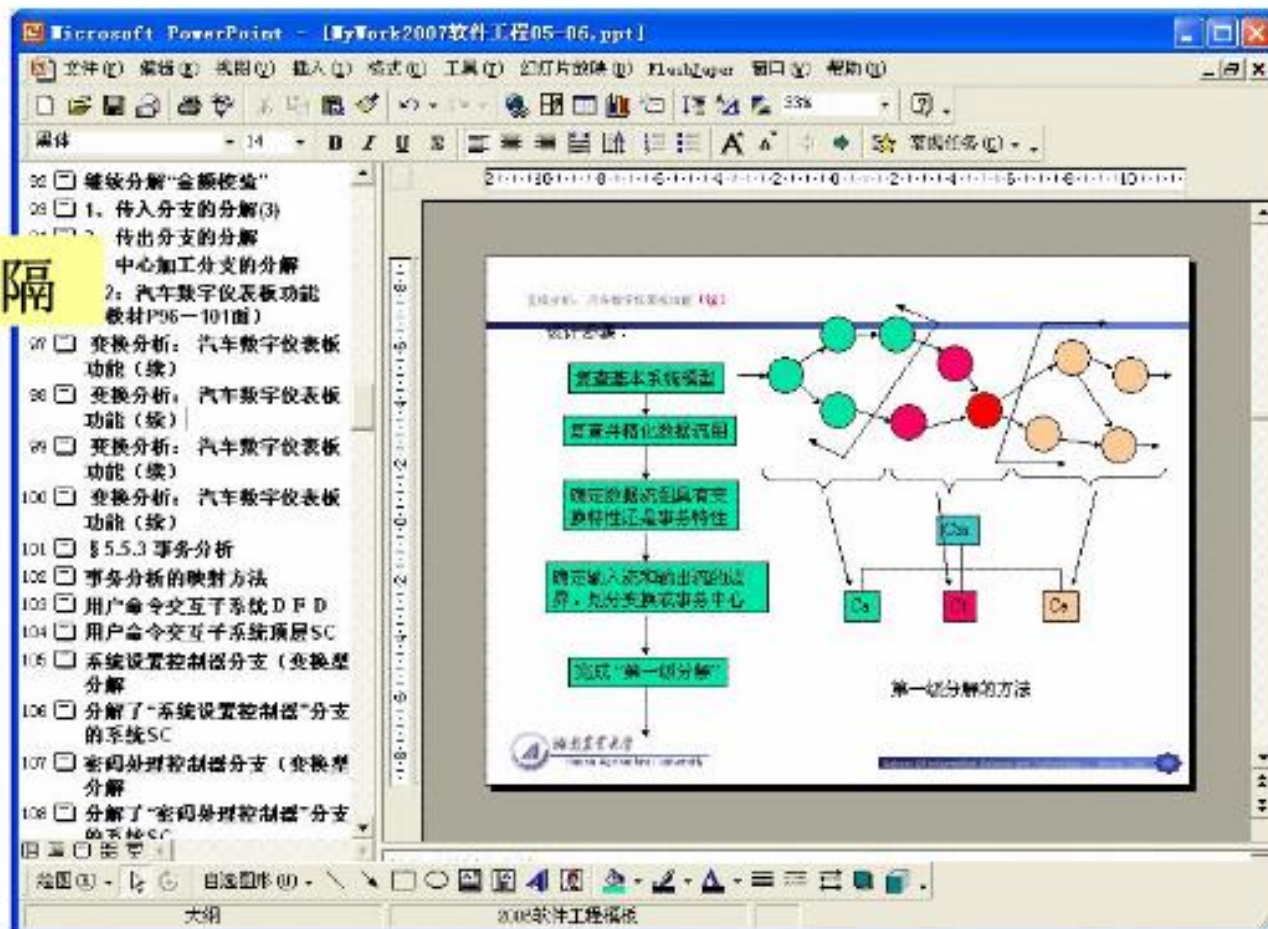
■ 基本设计指南

- 可用多种不同方式显示信息。如用图形表示数据
- 只显示与当前工作内容有关的信息
- 使用一致的标记、缩写和可预知的颜色。
- 使用窗口分隔不同类型的信息
- 使用模拟方式表示信息。

例：PowerPiont不同的界面显示方式

普通视图显示方式

用窗口分隔



界面举例网上红心大战

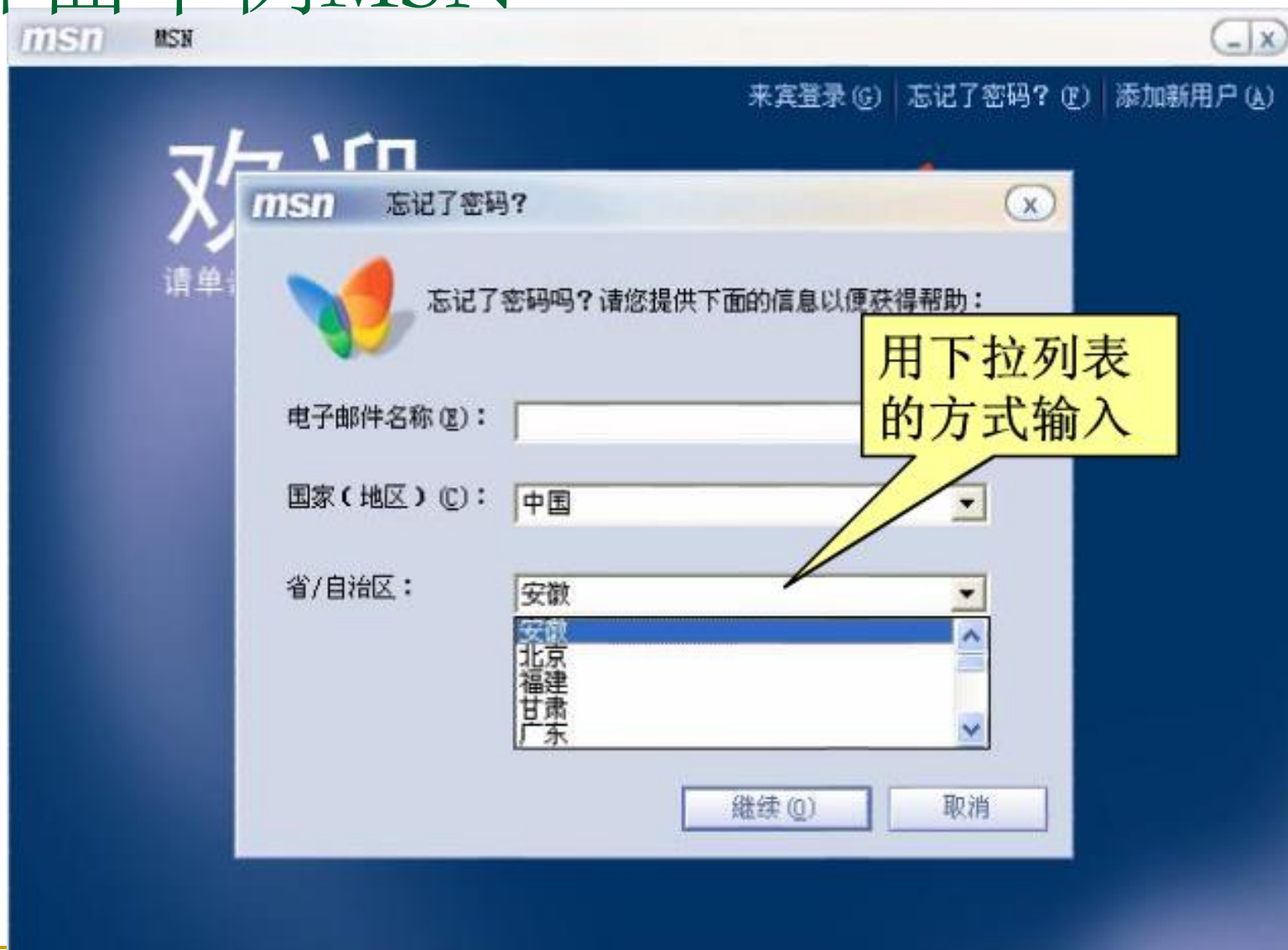


数据输入界面

■ 基本设计指南

- 尽量减少用户输入的动作
- 保持信息显示和数据输入的一致性
- 允许用户自定义输入，如键盘输入或鼠标输入
- 当前动作语境中，不适用的命令不起作用。
- 让用户控制交互流，如用户能跳过不必要的动作
- 消除冗余的输入

界面举例MSN



2.6 详细设计案例

详细设计书模板

1、SPM课程网站系统案例

SPM课程网站详细设计书

2、基于Web的电影院在线订票系统案例

基于Web的电影院在线订票系统设计
